

3/30/2026

Proiect 2

Aplicații ale Microcontrolerelor

Echipa 7

Studenti:

Criveanu Andra-Maria

Ioniță-Mitran Ioana

Profesor coordonator:

Zoican Sorin

Cuprins

Cerințe de proiectare	3
I. Codec	3
Codarea (secțiunea ADC - Analog-to-Digital Converter)	3
Decodarea (secțiunea DAC - Digital-to-Analog Converter).....	5
II. Automatic Gain Control (AGC)	6
III. Adaptive Line Enhancer (ALE).....	7
IV. Median Filter (MF)	8
V. Schema Electrică.....	8
Proiectarea.....	9
I. ARM Microcontroler	9
Introducere.....	9
Configurarea clockului	9
Timer 2.....	10
Pini de input.....	10
Pini de output	11
Generarea codului sursă.....	12
Setarea întreruperilor	13
II. DSP Microcontroler	14
Automatic Gain Control (AGC)	14
Adaptive line enhancer (ALE).....	16
Median Filter (MF)	18
Rezultate	20
III. Asamblarea funcțiilor într-un cod unic.....	25
Definirea parametrilor	25
Tabela vectori întreruperi	26
Inițializare sistem și filtre	27
Inițializare codec ad1847	27
ISR SPORT0: Citire și rutare comanda	28
ALGORITMUL AGC (Automatic Gain Control).....	30
ALGORITMUL ALE (Adaptive Line Enhancer)	32
ALGORITMUL FILTRU MEDIAN.....	34
SCRIERE IESIRE	35
ROUTINE AUXILIARE	36
IV. Setarea parametrilor funcției dsp conform stării switch-urilor de pe portul de intrare.....	38
Declararea adresei portului de intrare și valorilor maxime ale parametrilor	38
Inlocuirea parametrilor declarati #define cu variabile .var.....	38
Declararea unei variabile care stocheaza valoarea de la adresa portului de intrare	39
Prelucrarea valorii de pe portul de intrare pentru obținerea parametrilor	39
Modificări survenite în cod	42
V. Scriere la adresa portului de ieșire	43
Afișor cu 7 segmente	43
Declararea adresei portului de ieșire și vectorului de valori pentru segmentele afișorului	43

Implementarea și integrarea în cod a subrutinei de afișare	44
VI. Simulări	45
Adaptive Line Enhancer (ALE).....	45
Automatic Gain Control (AGC)	46
Median Filter (MF)	46
VII. Realizarea fizică a sistemului STM-DSP.....	49
STM32	49
Placă de extensie I/O ADSP2181	51

Cerințe de proiectare

Să se realizeze un sistem cu arhitectura din figura 1 cu următoarele specificații:

Microcontrolerul DSP va primi o comandă formată din 8 biți, aplicați pe porturile de tip Programmable Flags (configurate ca intrări), care dictează ce algoritm trebuie rulat.

- Biții D7 și D6 au rol de validare a comenzii, prin care DSP poate să se asigure că primește o comandă reală de la ARM la apariția unui impuls. Dacă acești 2 biți nu sunt amândoi 1, DSP va considera comanda invalidă.
- Biții D5 și D4 (CDA0 și CDA1) Pe baza lor, procesorul sare la subrutina corespunzătoare funcției.

(00) AGC

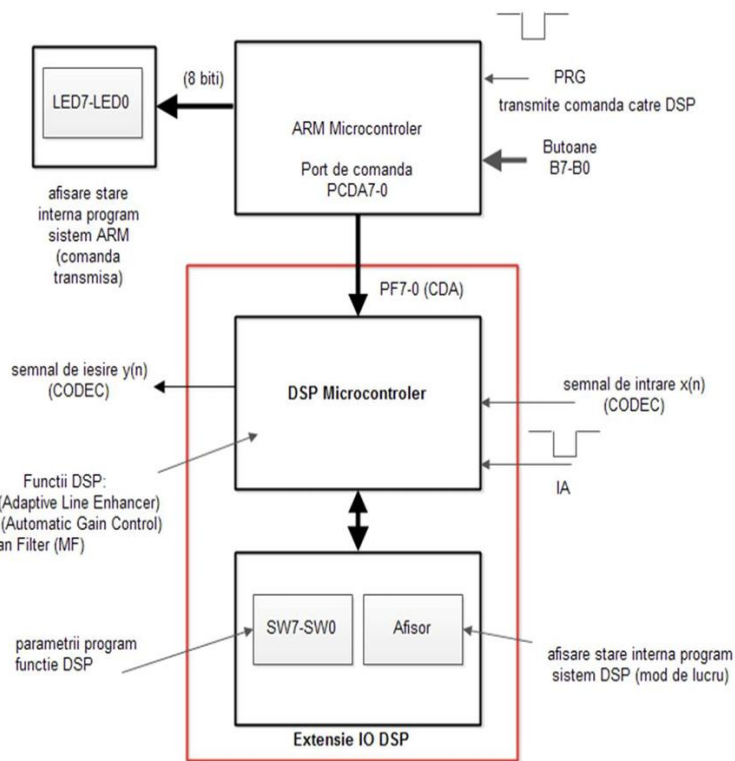
(01) ALE

(10) MF

(11) out = in (nu se execută nicio funcție)

- Bitul D3 (canal) selectează canalul semnalului pe care se aplică procesarea. El îi transmite DSP-ului dacă trebuie să prelucreze valorile de pe canalul stâng sau pe cele de pe canalul drept. Transferul automat al eșantioanelor între CODEC și memoria DSP-ului (rx_buf și tx_buf) se datorează autobuffering-ului, care generează o întrerupere periodică doar când bufferul este plin (pentru recepție) sau gol (pentru transmisie).
- Biții D2, D1, D0 (ID2, ID1, ID0) au rol de identificator al DSP-ului pe care se va executa comanda. Prin 3 biți putem alege din 8 DSP-uri diferite.

Intrarea $x(n)$ și ieșirea $y(n)$ sunt gestionate de CODEC prin intermediul portului serial SPORT0.



Figură 1

D7	D6	D5	D4	D3	D2	D1	D0
1	1	A1	A0	C	I2	I1	I0
Validare	Validare	Algoritm	Algoritm	Canal	ID DSP	ID DSP	ID DSP

I. Codec

CODEC-ul (Coder-Decoder) este termenul folosit pentru a reflecta dublul rol al acestuia în codarea datelor pentru recepție și transmisie, și decodarea lor ulterioară.

Codarea (secțiunea ADC - Analog-to-Digital Converter)

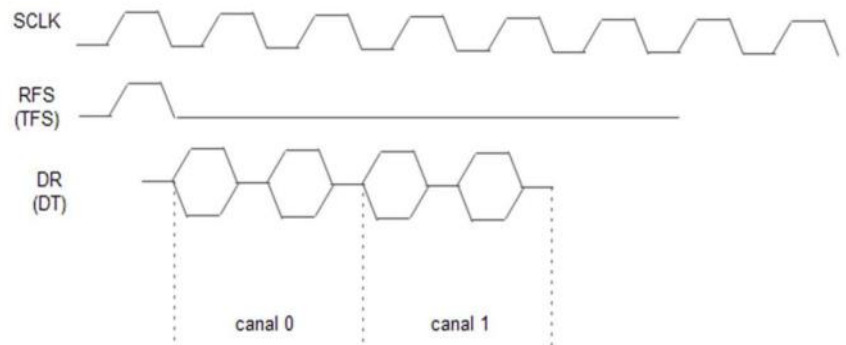
Preia semnalul electric analogic de intrare și îl măsoară la intervale regulate de timp (cu o rată de eșantionare dată de frecvența cu care este inițializat CODEC-ul), apoi transformă amplitudinea fiecărei măsurători într-un număr binar. Acest șir de numere reprezintă semnalul discret $x(n)$ pe care CODEC-ul îl trimite prin portul SPORT0 către DSP, ajungând în cele din urmă în bufferul tău de recepție rx_buf.

SPORT0 (Port Serial Sincron) este o interfață hardware specializată a DSP-ului, cu scopul de a transfera date de la CODEC către procesor fără a suprasolicita unitatea centrală.

SPORT0 prezintă 5 pini, iar pentru că este un port sincron, nu trimite numai date, ci și semnale de dirijare(ceas și sincronizare):

- SCLK: ceasul serial, fiecare front al semnalului dreptunghiular de ceas dictează momentul în care se scrie/citește un bit;
- RFS (Receive Frame Sync): un impuls scurt care comunică spre DSP începerea transmiterii unui nou eșantion de intrare;
- DR (Data Receive): pinul pe care intră bit cu bit valoarea eșantionului discret curent;

În spatele pinului DR, în interiorul DSP-ului, există un registru hardware. Pe măsură ce biții intră unul câte unul la fiecare tact de SCLK, ei sunt împinși în acest registru până când se formează un cuvânt complet, corespundent valorii eșantionului de intrare. Odată ce cuvântul a fost asamblat complet, SPORT folosește controlerul său DMA (Direct Memory Access) pentru a lua automat cuvântul de 16 biți și îl scrie direct în memoria RAM a DSP-ului, exact la adresa indicată de un registru de index. Tehnica multicanal înseamnă că la fiecare tact impus de CODEC, acesta nu trimite un singur eșantion, ci un pachet (cadru) care conține mai mulți biți, unul după altul: în proiect, într-un buffer de 3 locații: `.var/circ rx_buf[3]`.



- modul multicanal (un canal -> un grup de N biți) (de ex N=2)
- mod autobuffering (se incarca automat cuvintele receptionate intr-un buffer de receptie sau se descarca cuvintele, automat, dintr-un buffer de transmisie si se transmite -> intrerupere cind RxBuff e plin sau cind TxBuff e vid)



Conectarea DSP -codec- portul serial SPORT0

- La începutul pachetului, registrul de index arată spre `rx_buf + 0`, hardware-ul pune primul cuvânt acolo (un cuvânt de Status/Control al CODEC-ului) și incrementează automat registrul de index.
- Registrul de index arată spre `rx_buf + 1`, hardware-ul pune al doilea cuvânt în locația asociată canalului stânga și incrementează registrul de index.
- Acum, registrul de index arată spre `rx_buf + 2`, hardware-ul pune al treilea cuvânt (eșantionul util) în locația de memorie asociată canalului dreapta .

Fiind un buffer circular, hardware-ul resetează registrul de index înapoi la `rx_buf + 0` și declanșează întreruperea. Pentru că filtrele implementate sunt concepute în mod fundamental pentru a lucra pe un singur flux de date (semnal mono), din pachetul pe care CODEC îl transmite datorită protocolului său de comunicare, noi alegem să prelucrăm doar datele de pe `rx_buf+2` : `ar=dm(rx_buf+2)`;

- TFS (Transmit Frame Sync): un impuls pentru a comunica spre CODEC începerea unui nou eșantion de ieșire;
- DT(Data Transmit): pinul pe care iese bit cu bit valoarea eșantionului rezultat prin filtrare.

După ce procesorul a terminat de rulat algoritmul (AGC, ALE sau MF) pe eșantionul extras, rezultatul matematic este scris de către software într-o singură locație țintă din bufferul de transmisie, corespunzătoare canalului dorit (`dm(tx_buf + 2) = mr1;`). De aici, hardware-ul preia controlul complet pentru a trimite datele:

În interiorul DSP-ului, portul SPORT folosește controlerul său DMA (Direct Memory Access) pentru a extrage automat cuvintele de 16 biți din memoria RAM, de la adresele indicate de un registru de index de transmisie, fără a consuma timp de procesare. Conform tehnicii multicanal, SPORT-ul trebuie să trimită înapoi către CODEC un pachet (cadru) complet de 3 locații asociate bufferului `.var/circ tx_buf[3]`.

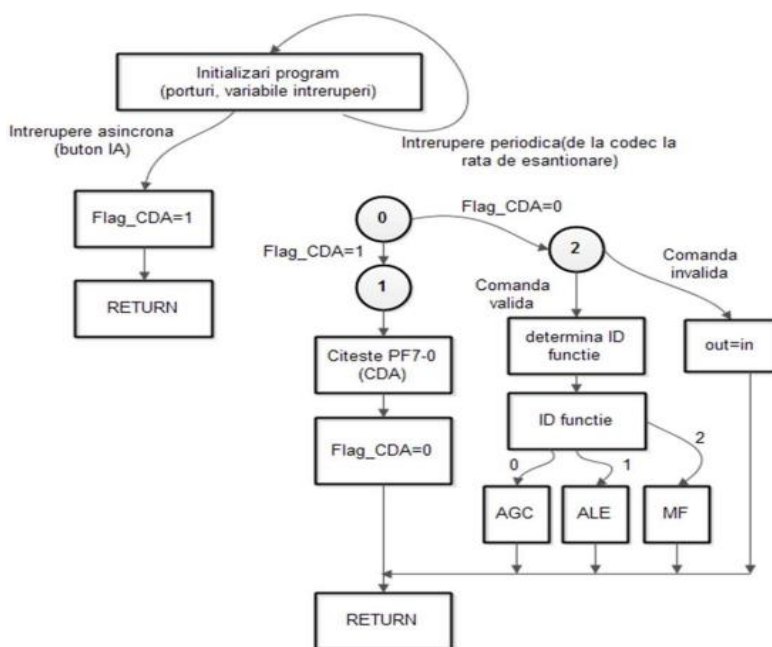
- La începutul pachetului (semnalizat de pinul TFS - Transmit Frame Sync), registrul de index arată spre tx_buf + 0. DMA-ul extrage cuvântul de aici (un cuvânt de Status/Control) și îl încarcă în registrul hardware de deplasare (Shift Register) al portului SPORT. Registrul de index se incrementează automat.
- Registrul de index arată acum spre tx_buf + 1. DMA-ul extrage al doilea cuvânt (asociat canalului stânga, pe care noi alegem să nu îl prelucrăm) și îl pregătește pentru transmisie. Registrul de index se incrementează.
- În final, registrul de index arată spre tx_buf + 2. DMA-ul extrage cuvântul de aici, care conține eșantionul filtrat util.
- Fiind un buffer circular, hardware-ul resetează apoi registrul de index înapoi la tx_buf + 0 (se declanșează întreruperea de transmisie când bufferul s-a golit complet).

În multe sisteme audio, SCLK, RFS și TFS sunt generate de CODEC (el este "Master"), iar DSP-ul doar ascultă și se conformează tempoului (este "Slave").

Decodarea (secțiunea DAC - Digital-to-Analog Converter)

După ce DSP-ul termină de rulat algoritmul selectat (AGC, ALE sau MF) pe acel eșantion, rezultă un nou număr binar, $y(n)$, plasat în bufferul de transmisie tx_buf. CODEC-ul preia acest număr și face procesul invers: "nivelează" aceste valori discrete pentru a reconstrui un semnal electric analogic continuu la ieșire.

Organigrama descrie logica executată în interiorul rutinelor de tratare a întreruperilor (ISR). Fluxul nu este o buclă infinită în programul principal, ci este condus de evenimente (întreruperi).



Organigramă DSP

Inițializări program : Se configurează porturile (PF-urile ca intrări, Afișorul ca ieșire), se inițializează bufferele circulare de recepție și transmisie (rx_buf și tx_buf), începe activitatea CODEC și se activează întreruperile.

Întreruperea Asincronă (butonul IA) semnalizează DSP-ului că o nouă comandă este disponibilă pe pinii PF. În urma apăsării butonului, procesorul se oprește brusc din prelucrare, setează variabila Flag_CDA=1, apoi revine la prelucrare (**RETURN**).

Fluxul principal se întâmplă la întreruperea periodică declanșată de umplerea bufferului rx_buf de către CODEC prin SPORT.

Declanșarea periodică (STAREA 0): CODEC a terminat de transferat pachetul de cuvinte, procesorul intră în rutină și ajunge la un punct de decizie (Nodul 0):

Valoarea **Flag_CDA=1** determină trecerea în **STAREA 1 (Schimbarea Comenzii)**: flagul setat înștiințează procesorul că trebuie să preia o comandă nouă. Se citesc biții de comandă de la portul fizic PF7-0 și se memorează valoarea. Flagul Flag_CDA se resetează și se execută **RETURN**.

Valoarea **Flag_CDA=0** determină trecerea în **STAREA 2 (Procesarea Semnalului)**: se verifică biții de validare ai comenzii curente memorate (D7 și D6 trebuie să aibă ambii valoarea 1 pentru validare):

Comandă validă: Se determină ID-ul DSP-ului pe baza valorii biților D2, D1 și D0 din comanda memorată.

În interiorul subrutinei funcției selectate, algoritmul extrage eșantionul util (rx_buf+2), citește switch-urile hardware SW7-SW0 pentru a ajusta parametrii (ex: pasul de adaptare μ), face calculele matematice și actualizează Afișorul (indică ID-ul filtrului care se aplică în prezent).

Rezultatul este plasat în tx_buf+2, apoi se execută **RETURN**. CODEC-ului îi rămâne de preluat rezultatul și de revenit la un semnal analogic din cel obținut prin filtrare.

Comandă invalidă: Se execută **out=in**. DSP citește ar=dm(rx_buf+2) și scrie dm(tx_buf+2)=ar.

Tot pe această ramură, este recomandată stingerea Afișorului (sau afișarea unui cod de eroare) pentru a nu păstra o valoare reziduală falsă, indicând utilizatorului că sistemul a intrat pe ramura de comandă invalidă.

II. Automatic Gain Control (AGC)

În principiu, un AGC este un sistem de control cu buclă de reacție care reduce eroarea de amplitudine la zero într-un mod iterativ. Acesta stabilește o amplitudine constantă a semnalului la începutul lanțului DSP.

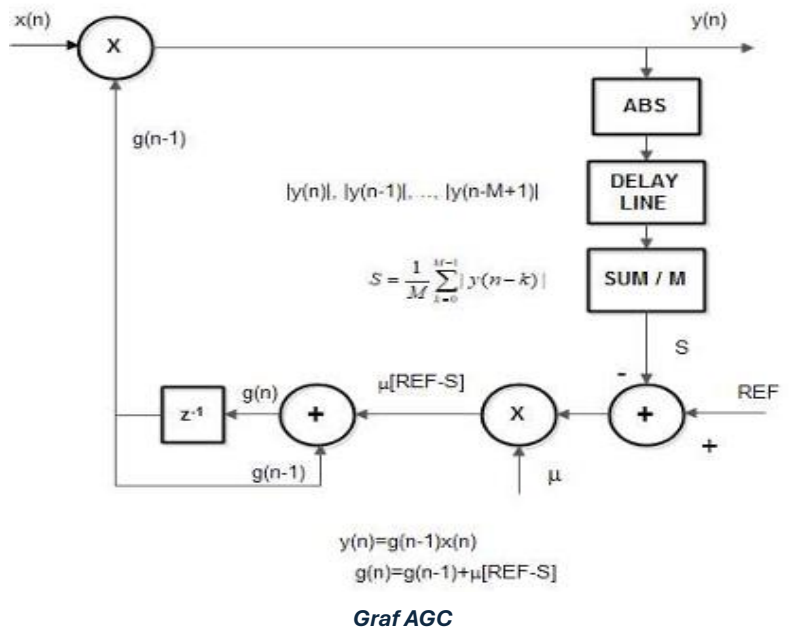
Semnalul de intrare $x(n)$ este înmulțit cu ieșirea AGC $a(n)$, producând astfel ieșirea $z(n)$. În bucla de feedback, amplitudinea semnalului complex $z(n)$ se găsește ca $|z(n)| = |g(n-1) \cdot x(n)|$.

Nivelul de referință R este nivelul unde dorim ca ieșirea $z(n)$ să se stabilizeze după convergența AGC. Pentru a îndeplini această sarcină, se produce un semnal de eroare ca diferența dintre referința R și media celor M eșantioane: $e(n) = R - |g(n-1) \cdot x(n)|$.

Ajustarea amplitudinii va fi în funcție de un parametru μ . Valoarea parametrului μ dictează viteza de învățare a filtrului, trebuie aleasă astfel încât să nu fie nici prea mare, cât sistemul să nu devină instabil, și nici prea mică, pentru o rată de adaptare cu utilizare eficientă a benzii.

$$g[n] = g[n-1] + \mu \cdot e[n]$$

Expresia obținută în urma acumulării ultimului coeficient cu eroarea înmulțită cu coeficientul μ realizează ajustarea coeficientului și este foarte similară cu ecuația de actualizare a celor mai mici pătrate (LMS), care este un caz special al unui algoritm adaptiv. Atunci când nivelul semnalului de intrare este ridicat, se transmite o tensiune negativă, reducând astfel amplificarea care scade nivelul semnalului după multiplicare. O operație similară se efectuează pentru a crește amplificarea atunci când nivelul semnalului de intrare este scăzut.



III. Adaptive Line Enhancer (ALE)

Adaptive Line Enhancer (ALE) este o tehnică de procesare a semnalului utilizată pentru a separa și amplifica semnalele, în prezența zgomotului, prin utilizarea unor metode de filtrare adaptivă. Un ALE se bazează pe conceptul de predicție liniară a formei semnalului. Un semnal aproape periodic poate fi prezis perfect folosind combinații liniare ale eșantioanelor sale anterioare, în timp ce un semnal neperiodic (aleator) nu poate.

Semnalul rezultat $xz(n)=x(n)+a \cdot z(n)$ este suma semnalului dorit (periodic) cu zgomot (aleator).

O versiune întârziată a semnalului măsurat $xz(n-D)$ este utilizată ca semnal de intrare de referință pentru filtrul adaptiv. Zgomotul aplicat peste semnalul util este aleatoriu, deci valorile sale prezente nu sunt corelate cu valorile sale trecute. Semnalul periodic, în schimb, se repetă. O valoare de acum este foarte strâns legată de o valoare de acum D eșantioane. Ieșirea blocului de întârziere conține un semnal periodic care **este** corelat cu intrarea curentă, și un zgomot care **nu este** corelat cu zgomotul de la intrarea curentă.

Ecuția pentru ieșirea filtrului este ecuația standard a unui filtru FIR (Finite Impulse Response):

$$y[n] = \sum_{k=0}^{N-1} h(k) \cdot xz(n-k-D)$$

Deoarece doar componenta periodică a semnalului întârziat este corelată cu semnalul curent nestrecurat, filtrul va învăța să lase să treacă și să amplifice doar acea componentă periodică, ignorând zgomotul (pe care nu îl poate prezice). Astfel, ieșirea $y(n)$ devine o estimare curată a semnalului util.

Eroarea se calculează ca diferența dintre semnalul brut curent și predicția făcută de filtru:

$$e[n] = xz[n] - y(n)$$

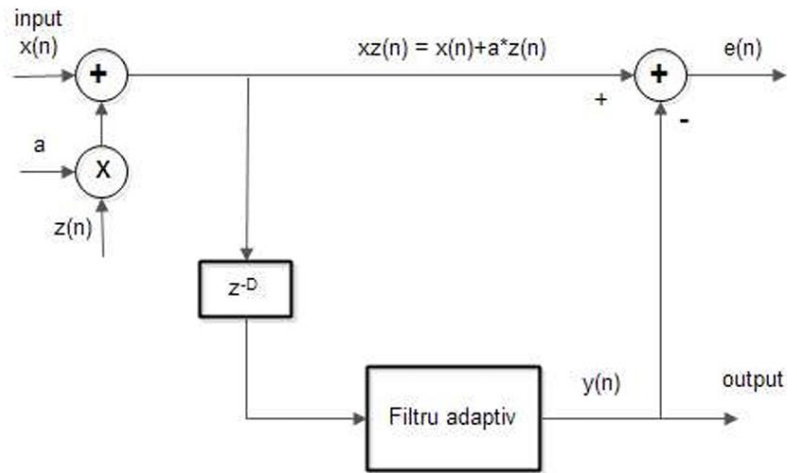
Această eroare este folosită ca semnal de feedback pentru a ajusta coeficienții filtrului. După convergență, eroarea $e(n)$ va conține doar zgomotul imprevizibil pe care filtrul nu a reușit să îl prezică.

Ecuția ne arată cum își actualizează filtrul coeficienții $h(k)$ de adaptare:

$$h(k) = (1 - \lambda\mu)h(k) + \mu xz(n-k-D)e(n), k = 0, 1, \dots, N-1$$

μ dictează viteza de învățare a filtrului adaptiv. Rolul termenului $(1-\mu\lambda)$ este să prevină creșterea infinită a coeficienților în cazul în care la intrare există doar zgomot alb, asigurând stabilitatea sistemului pe termen lung.

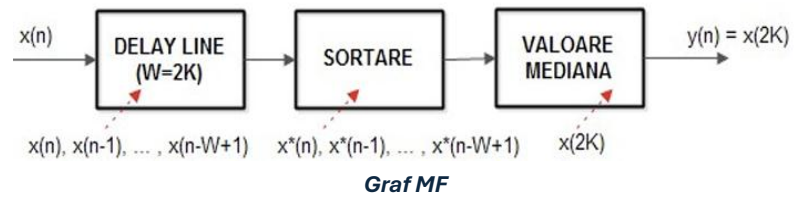
Coeficienții $h(k)$ au valoarea inițială 0. La momentul pornirii, sistemul nu are nicio informație despre frecvența semnalului periodic sau despre zgomot. Pornind de la zero, filtrul nu face nicio presupunere greșită care ar putea prelungi timpul de învățare (convergență). Când toți coeficienții sunt 0, ieșirea filtrului la primul pas este evident 0. Această eroare mare forțează algoritmul să modifice imediat coeficienții și să înceapă adaptarea.



Graf ALE

IV. Median Filter (MF)

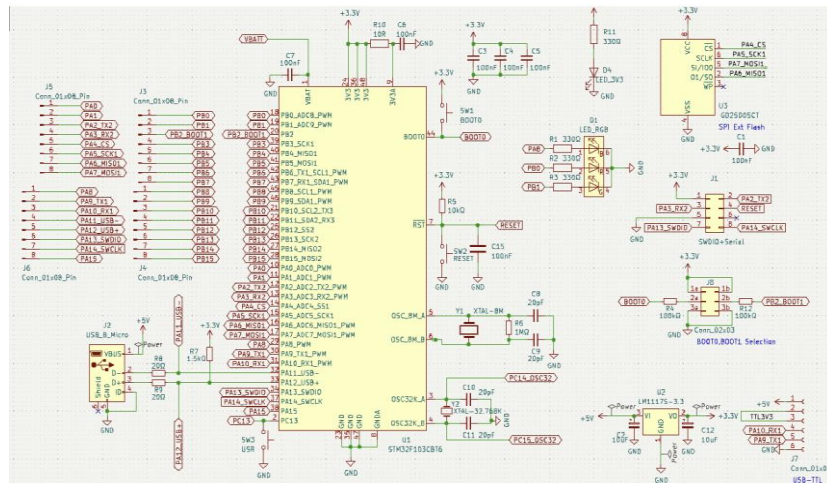
Filtrul median este clasificat ca o tehnică neliniară de filtrare digitală, deoarece ieșirea sa nu este o funcție liniară a datelor de intrare. Filtrul median funcționează prin înlocuirea valorii unui punct de date cu valoarea mediană găsită în vecinătatea sa locală. Această operație fundamentală îl face robust împotriva zgomotului ce se manifestă



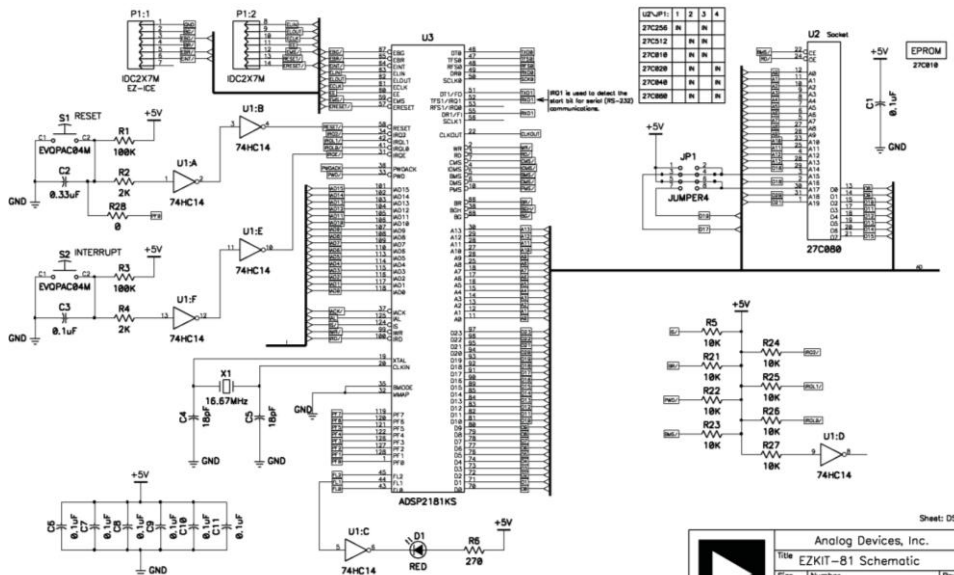
Procesul începe cu o „fereastră glisantă” (Delay Line), care definește vecinătatea locală utilizată pentru calcul. Această fereastră se mișcă sistematic pe întreaga gamă de date de intrare, eșantion cu eșantion. Valorile sunt apoi sortate numeric (crescător sau descrescător), proces reprezentat prin blocul SORTARE, creând o listă ordonată a conținutului vecinătății. Fereastra are o dimensiune de valoare impară $W=2K+1$, pentru a putea regăsi cea valoare mediană aflată în mijlocul setului de date.

Pasul următor este selectarea medianei, valoarea care se află exact în mijlocul acestei liste sortate. Când fereastra glisantă întâlnește una dintre aceste valori extreme de zgomot, procesul de sortare plasează valoarea la un capăt al listei. Valoarea mediană devine apoi eșantionul curent al semnalului de ieșire $y(n)$.

V. Schema Electrică



ARM Microcontroller



DSP Microcontroler

Proiectarea

I. ARM Microcontroller

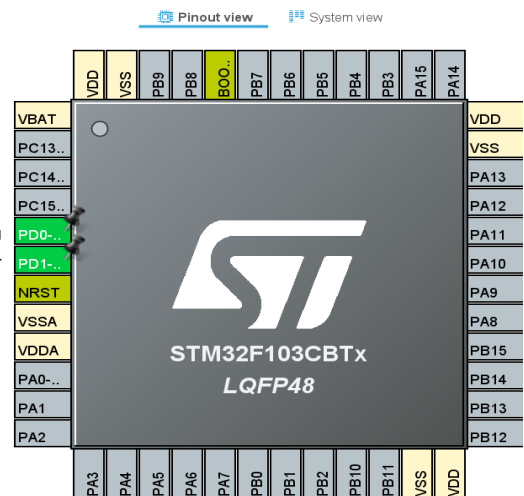
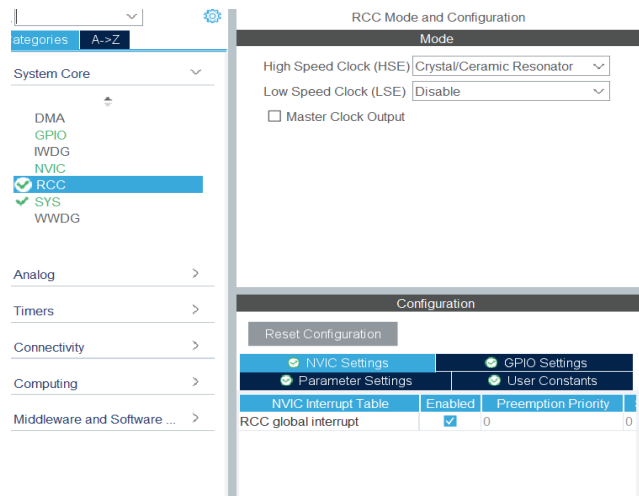
Introducere

Proiectul este bazat pe un microcontroller din familia STM32F1, mai exact modelul STM32F103CBTx. Acesta dispune de un procesor Arm Cortex-M3 și folosește o capsulă de tip LQFP48 (care are 48 de pini).

Configurarea clockului

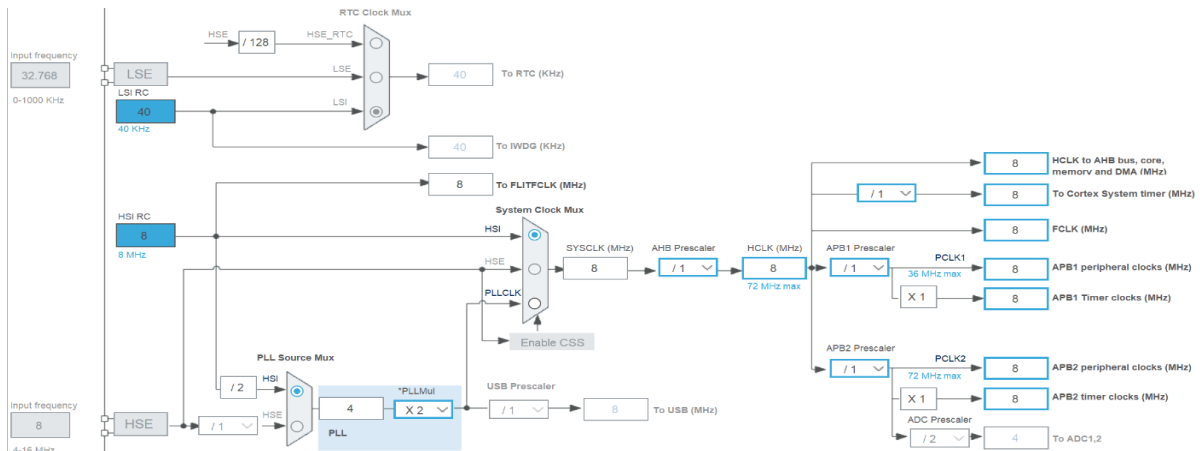
Pinii alocați clockului sunt: PD0-OSC_IN și PD1-OSC_OUT, aceștia se colorează în verde după ce se fac setările corespunzătoare.

Intrăm în modul RCC(Reset and Clock Control) pentru a seta oscilatorul extern(High Speed Clock) pe modul „Crystal/Ceramic Resonator”. Asta înseamnă că îi spun microcontrollerului să folosească o componentă hardware externă (un cuarț/rezonator) pe post de sursă principală de ceas, în loc de oscilatorul său intern.



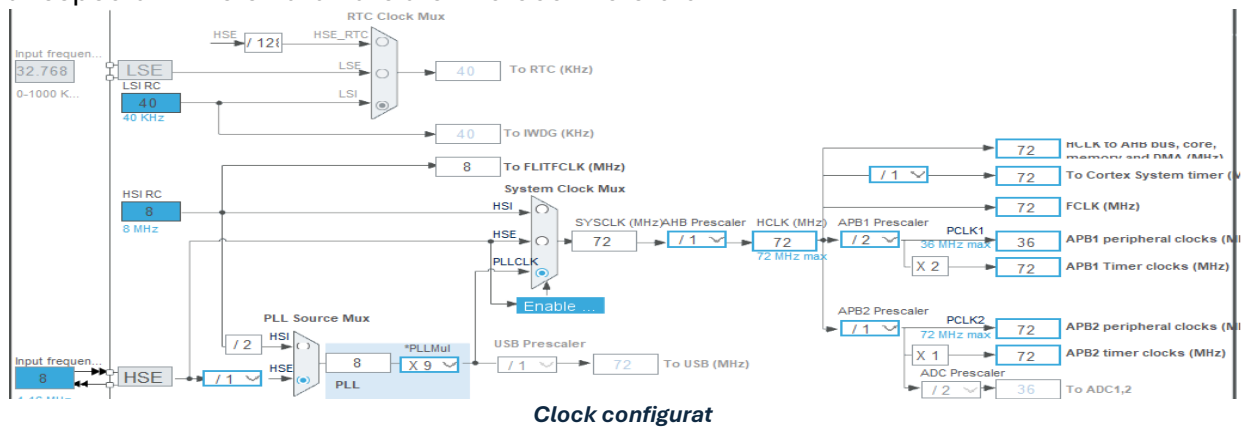
Pași configurare

Modificările principale au constat în trecerea de la oscilatorul intern (HSI) la un oscilator extern precis (HSE de 8 MHz), urmată de activarea multiplicatorului PLL (setat la x9) pentru a crește viteza procesorului de la valoarea de bază de 8 MHz direct la capacitatea sa maximă de 72 MHz. Odată cu această accelerare a întregului sistem, a trebuit neapărat să ajustăm și divizorul (prescaler-ul) magistralei



Clock neconfigurat

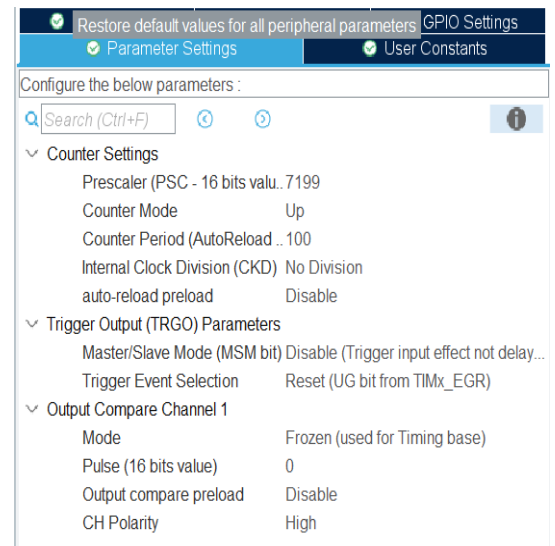
APB1 la valoarea /2, limitând astfel frecvența la 36 MHz pentru a menține stabilitatea perifericelor mai lente și a respecta limitele hardware ale microcontrolerului.



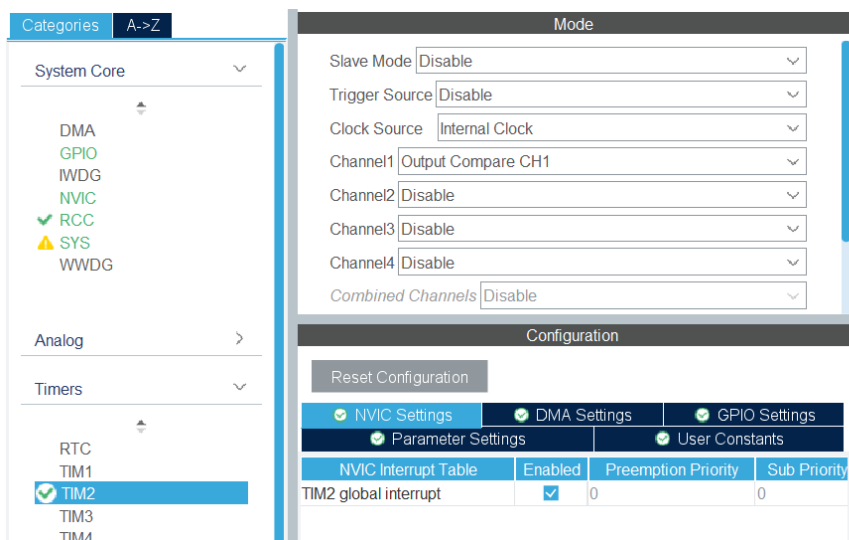
Timer 2

Sursa de ceas a fost setată pe Internal Clock, iar Canalul 1 (Channel1) a fost activat în modul Output Compare CH1. Această acțiune a asociat automat pinul PA0 cu funcția timerului. Pentru a încetini ceasul rapid al sistemului la un interval de timp măsurabil și util, s-a aplicat un divizor Prescaler (PSC) cu valoarea de 7199 și o limită de numărare Counter Period (AutoReload) setată la 100. Modul ieșirii a fost setat pe Frozen (used for Timing base), ceea ce înseamnă că timerul este folosit exclusiv pentru a măsura timpul intern, fără a comuta fizic semnalul pe pinul PA0.

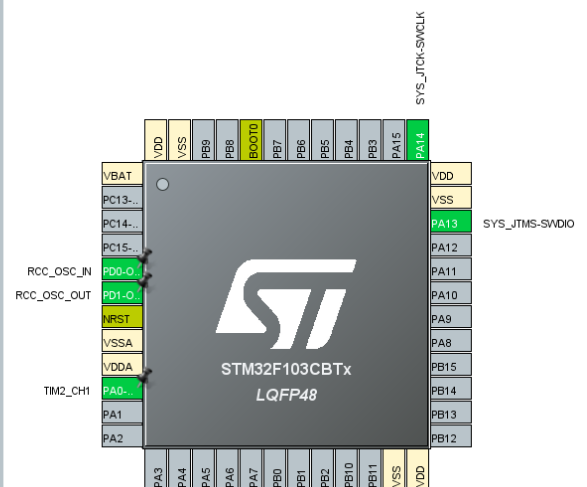
În secțiunea NVIC Settings, a fost bifată opțiunea TIM2 global interrupt. Prin acest pas esențial, microcontrolerul va genera o întrerupere de fiecare dată când timerul ajunge la limita setată, permițând codului să execute o acțiune la intervale de timp precise și regulate.



TIM2 Mod și configurare



TIM2 Mod și configurare



Pini de input

Pentru a defini un pin ca o intrare digitală capabilă să citească starea unui buton am făcut click pe un din reprezentarea grafică a microcontrolerului și am selectat din meniu GPIO_Input, astfel pentru cele 8 butoane am setat pinii **PA1** până la **PA8**. Aceștia aveau configurația implicită: modul de intrare ("Input mode") și fără rezistențe interne de pull-up sau pull-down activate ("No pull-up and no pull-down").

Pentru a face codul mai ușor de înțeles și de gestionat, pinii au fost redenumiți cu etichete personalizate, astfel pinilor de la **PA1** la **PA8** li s-au atribuit etichetele de la **B0** la **B7**. Aceste denumiri (User Labels) permit referirea la pini prin nume logice în codul sursă, în loc de denumirile hardware generice.

Pinul **PA9** este tot input pentru microcontroler pentru că el este cel care determina trimiterea biților de comanda de la STM la DSP, printr-un impuls negativ, denumirea pentru acest pin este **PRG**.

Pi...	Signal	GPIO	GPIO	GPIO	Maxi...	User	Modifi...
PA1	n/a	n/a	Input ...	No pul...	n/a	B0	✓
PA2	n/a	n/a	Input ...	No pul...	n/a	B1	✓
PA3	n/a	n/a	Input ...	No pul...	n/a	B2	✓
PA4	n/a	n/a	Input ...	No pul...	n/a	B3	✓
PA5	n/a	n/a	Input ...	No pul...	n/a	B4	✓
PA6	n/a	n/a	Input ...	No pul...	n/a	B5	✓
PA7	n/a	n/a	Input ...	No pul...	n/a	B6	✓
PA8	n/a	n/a	Input ...	No pul...	n/a	B7	✓
PA9	n/a	n/a	Input ...	No pul...	n/a	PRG	✓

Tabel GPIO Input

Pini de output

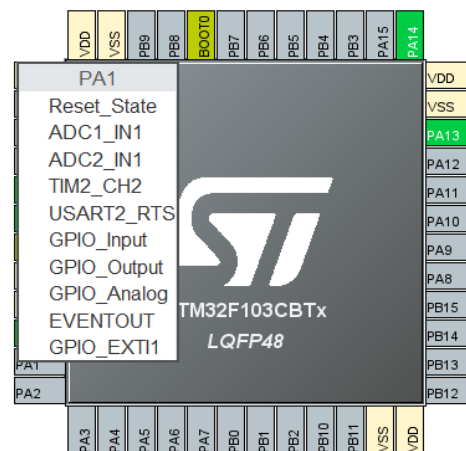
Pentru a defini pinii ca ieșiri digitale capabile să transmită instrucțiunea prelucrată de automat selectăm din meniu GPIO_Output, am folosit portul B pentru a-i configura.

Led

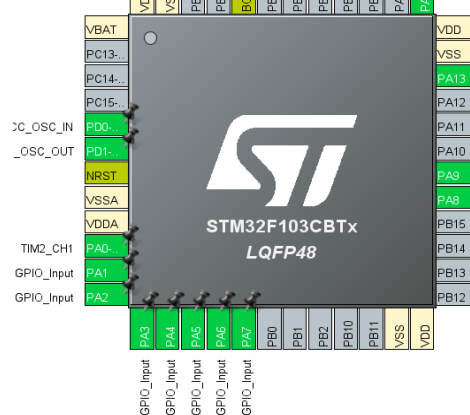
Pinilor de la **PB0** la **PB7** li s-au atribuit etichetele de la **LED0** la **LED7**, pentru că o să îi folosim ca leduri pentru a afișa starea internă a programului (comanda transmisă).

PCDA (Parallel Character Data Array)

Pinilor de la **PB8** la **PB15** li se atribuie etichetele de la **PCDA0** la **PCDA7**, pentru a defini o magistrală de date de 8 biți care comunică comanda la DSP



Meniul de configurare a pinilor



Pinii înainte de etichete

PB0	n/a	Low	Outpu...	No pul...	Low	LED0	✓
PB1	n/a	Low	Outpu...	No pul...	Low	LED1	✓
PB2	n/a	Low	Outpu...	No pul...	Low	LED2	✓
PB3	n/a	Low	Outpu...	No pul...	Low	LED3	✓
PB4	n/a	Low	Outpu...	No pul...	Low	LED4	✓
PB5	n/a	Low	Outpu...	No pul...	Low	LED5	✓
PB6	n/a	Low	Outpu...	No pul...	Low	LED6	✓
PB7	n/a	Low	Outpu...	No pul...	Low	LED7	✓

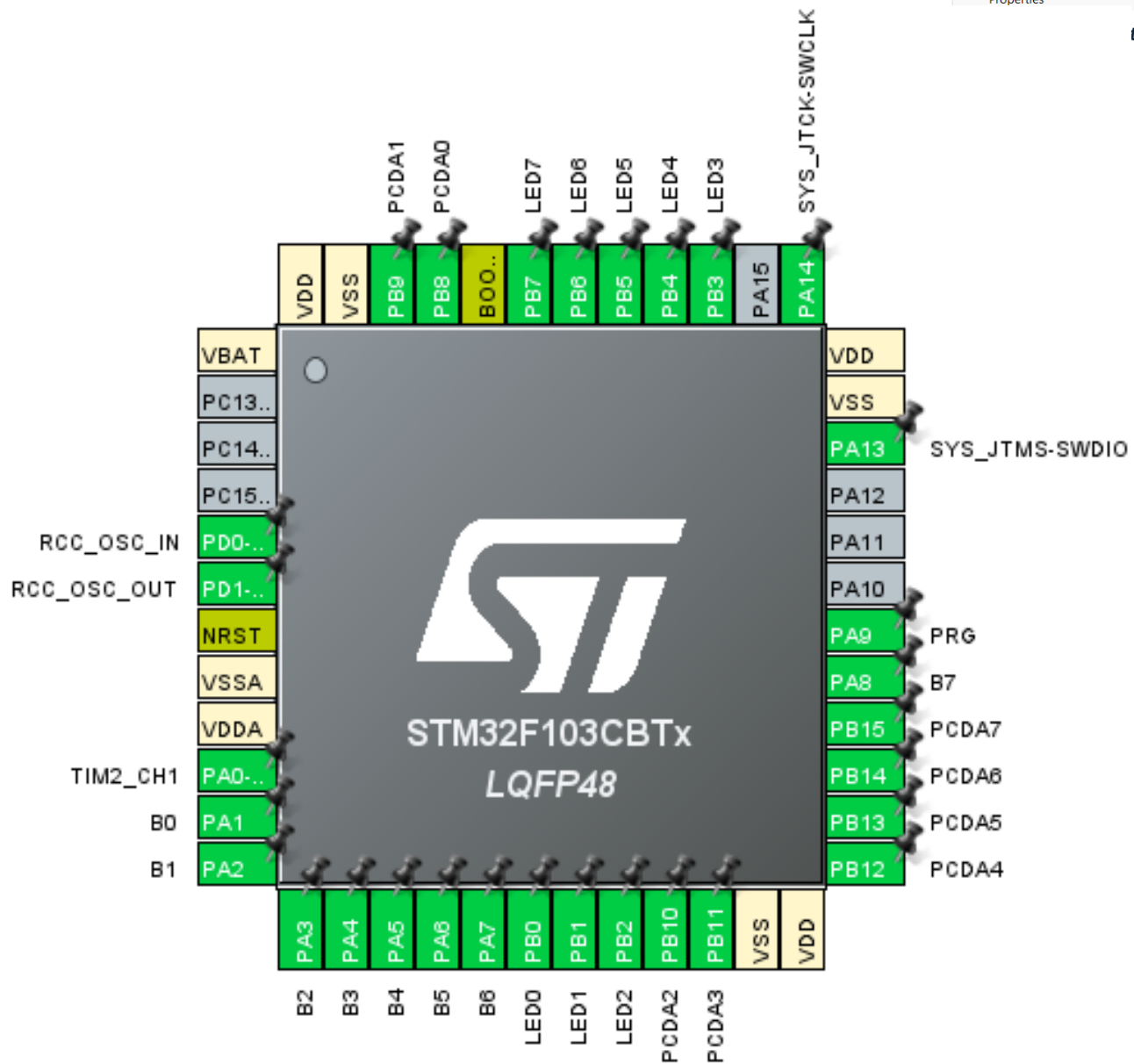
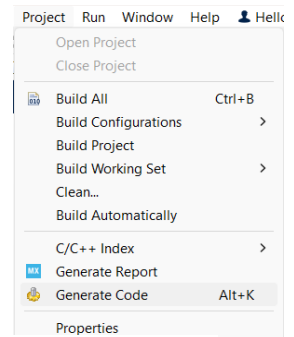
Tabel GPIO_Output Leduri

PB8	n/a	Low	Outpu...	No pul...	Low	PCDA0	✓
PB9	n/a	Low	Outpu...	No pul...	Low	PCDA1	✓
PB10	n/a	Low	Outpu...	No pul...	Low	PCDA2	✓
PB11	n/a	Low	Outpu...	No pul...	Low	PCDA3	✓
PB12	n/a	Low	Outpu...	No pul...	Low	PCDA4	✓
PB13	n/a	Low	Outpu...	No pul...	Low	PCDA5	✓
PB14	n/a	Low	Outpu...	No pul...	Low	PCDA6	✓
PB15	n/a	Low	Outpu...	No pul...	Low	PCDA7	✓

Tabel GPIO_Output PCDA

Generarea codului sursă

După ce toate intrările, ieșirile și funcțiile speciale au fost definite putem să accesăm meniul „Project” opțiunea „Generate Code”, care va citi toate setările și va genera automat fișierele C care conțin scheletul de bază al programului.



Placa configurată

Setarea întreruperilor

Am verificat dacă rutina generală de servire a întreruperilor a fost pusă automat în fișierul C **stm32f1xx_it.c**.

Am pornit timerul 2 cu ajutorul comenzii **HAL_TIM_Base_Start_IT(&htim2);** din secțiunea **USER CODE BEGIN 2**, care generează o întrerupere la fiecare 10 milisecunde, astfel se va opri așteptarea infinită a procesorului dată de bucla **while(1)** până la apăsarea butonului.

Am adăugat în **main.c** la secțiunea **USER CODE BEGIN 0** funcția **HAL_TIM_PeriodElapsedCallback**, care va fi apelată de funcția **HAL_TIM_IRQHandler** din fișierul **stm32f1xx_hal_tim.c**.

Funcția **HAL_TIM_PeriodElapsedCallback** este cea care descrie automatul de „Servire întreruperi periodice SCI” din schema bloc a MCU-ului. La fiecare declanșare a timerului 2, funcția va parcurge următoarele stări:

Blocul 1 (Citirea datelor):

Programul citește fizic starea pinului de comandă (PRG) și a celor 8 butoane (B0-B7) folosind instrucțiunea **HAL_GPIO_ReadPin**, astfel se salvează starea curentă a sistemului.

Blocul 2 (Automatul de stări/TRS_CDA):

Detectează un impuls complet (apăsare + eliberare), astfel se asigură că indiferent de cât de mult este apăsat butonul PRG comanda se va transmite o singură dată. Acest lucru este implementat prin variabilei **Q**, care reține starea curentă și structura **switch(Q)**:

Starea 0: Verifică dacă pinul PRG este RESET și trece la Starea 1 dacă este.

Starea 1: Verifică dacă PRG a devenit SET și trece la Starea 2 dacă este.

Starea 2: Aici se copiază valorile biților citiți din pinii de ieșire (PCDA=comanda), iar variabila **Q** revine la Starea 0.

Blocul 3 (Return):

Programul iese automat din funcția de întrerupere și așteaptă următoarea întrerupere generată de timer.

```
1.c | Projctioc | stm32f1xx_it.c x
/* USER CODE BEGIN SysTick_IRQn 0 */
HAL_IncTick();
/* USER CODE BEGIN SysTick_IRQn 1 */

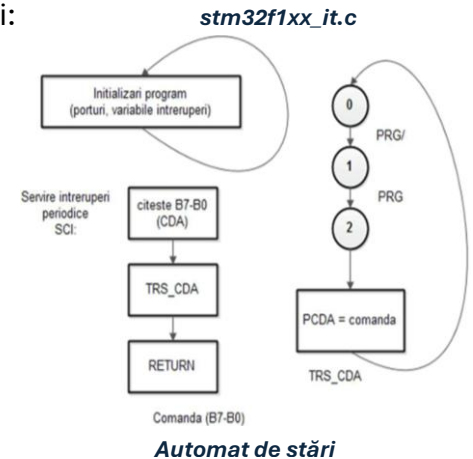
/* USER CODE END SysTick_IRQn 1 */

/*****
 * STM32F1xx Peripheral Interrupt Handlers
 * Add here the Interrupt Handlers if
 * For the available peripheral interrupt
 * please refer to the startup file
 *****/

/**
 * @brief This function handles TIM2
 */
void TIM2_IRQHandler(void)
{
/* USER CODE BEGIN TIM2_IRQn 0 */

/* USER CODE END TIM2_IRQn 0 */
HAL_TIM_IRQHandler(&htim2);
/* USER CODE BEGIN TIM2_IRQn 1 */

/* USER CODE END TIM2_IRQn 1 */
}
```



```
main.c x | Projctioc
60=void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim2)
61 {
62     if (htim2->Instance == TIM2)
63     {
64         // Variabila de stare Q pentru automat (trebuie sa fie statica pentru a-si pastra valoarea)
65         static uint8_t Q = 0;
66
67         // Variabile pentru a stoca starea curenta a butoanelor
68         GPIO_PinState b0, b1, b2, b3, b4, b5, b6, b7;
69         GPIO_PinState prg_state;
70
71         // 1. BLOCUL: "citeste B7-B0 (CDA)" + citire PRG
72         prg_state = HAL_GPIO_ReadPin(GPIOA, PRG_Pin);
73
74         b0 = HAL_GPIO_ReadPin(GPIOA, B0_Pin);
75         b1 = HAL_GPIO_ReadPin(GPIOA, B1_Pin);
76         b2 = HAL_GPIO_ReadPin(GPIOA, B2_Pin);
77         b3 = HAL_GPIO_ReadPin(GPIOA, B3_Pin);
78         b4 = HAL_GPIO_ReadPin(GPIOA, B4_Pin);
79         b5 = HAL_GPIO_ReadPin(GPIOA, B5_Pin);
80         b6 = HAL_GPIO_ReadPin(GPIOA, B6_Pin);
81         b7 = HAL_GPIO_ReadPin(GPIOA, B7_Pin);
82
83         // 2. BLOCUL: "TRS_CDA" (Automatul de stări)
84         switch (Q)
85         {
86             case 0:
87                 // Așteptăm apăsarea PRG (PRG -> LOW / RESET)
88                 if (prg_state == GPIO_PIN_RESET)
89                 {
90                     Q = 1; // Treceam în starea 1
91                 }
92                 break;
93             case 1:
94                 // Așteptăm eliberarea PRG (PRG -> HIGH / SET)
95                 if (prg_state == GPIO_PIN_SET)
96                 {
97                     Q = 2; // Treceam în starea 2
98                 }
99                 break;
100         }
```

```
main.c x | Projctioc
101
102 case 2:
103     // BLOCUL: "PCDA = comanda"
104     // Scriem valorile citite pe portul PCDA și pe LED-uri
105     HAL_GPIO_WritePin(GPIOB, LED0_Pin, b0);
106     HAL_GPIO_WritePin(GPIOB, PCDA0_Pin, b0);
107
108     HAL_GPIO_WritePin(GPIOB, LED1_Pin, b1);
109     HAL_GPIO_WritePin(GPIOB, PCDA1_Pin, b1);
110
111     HAL_GPIO_WritePin(GPIOB, LED2_Pin, b2);
112     HAL_GPIO_WritePin(GPIOB, PCDA2_Pin, b2);
113
114     HAL_GPIO_WritePin(GPIOB, LED3_Pin, b3);
115     HAL_GPIO_WritePin(GPIOB, PCDA3_Pin, b3);
116
117     HAL_GPIO_WritePin(GPIOB, LED4_Pin, b4);
118     HAL_GPIO_WritePin(GPIOB, PCDA4_Pin, b4);
119
120     HAL_GPIO_WritePin(GPIOB, LED5_Pin, b5);
121     HAL_GPIO_WritePin(GPIOB, PCDA5_Pin, b5);
122
123     HAL_GPIO_WritePin(GPIOB, LED6_Pin, b6);
124     HAL_GPIO_WritePin(GPIOB, PCDA6_Pin, b6);
125
126     HAL_GPIO_WritePin(GPIOB, LED7_Pin, b7);
127     HAL_GPIO_WritePin(GPIOB, PCDA7_Pin, b7);
128
129     // Revenim în starea inițială
130     Q = 0;
131     break;
132 }
133 // 3. BLOCUL: "RETURN" (Se iese automat din funcția de întrerupere)
134 }
135
136 /* USER CODE END 0 */
```

Cod automat secvențial

II. DSP Microcontroller

DSP are registre strict specializate, grupate în jurul unor unități hardware distincte, pentru a putea face mai multe lucruri în același ciclu de ceas, astfel registrele generale din familia ADSP-218x sunt:

Registrele unității de calcul

Unitatea aritmetică logică (ALU): adunări, scăderi, operații logice și funcții absolute

- AX0, AX1, AY0, AY1: intrări pe 16 biți ale unității ALU
- AR (ALU Result): ieșirea de 16 biți unde se salvează rezultatul
- AF (ALU Feedback) folosit pentru a reține rezultatul intern ca să seteze un flag

Unitatea MAC (Multiplier-Accumulator): filtre digitale și transformata Fourier

- MX0, MX1, MY0, MY1: intrări pe 16 biți pentru înmulțire
- MR0, MR1, MR2 (MAC Result): formează un acumulator gigant pe 40 de biți (MR0-16, MR1-16, MR2-8)
- MF (MAC Feedback): păstrează rezultatul multiplicatorului fără a scrie în acumulator

Unitate de Shiftare (Shifter): manipulează biții pentru scalare sau extragere unor părți

- SI: intrare pe 16 biți
- SE (Shift Exponent): registru pe 8 biți care îi spune hardware-ului cu câte poziții să mute biții
- SR0, SR1 (Shift Result): ieșiri de 16 biți

Registrele generatorilor de adrese (DAG)

- Registrele I (Index) pointeri care rețin adresa exactă din memorie
- Registrele M (Modify) indică pasul de deplasare
- Registrele L (Length) setează limitele bufferelor circulare, se întoarce singur la început

Registrele de control și status

- PC (Program Counter) ține minte adresa instrucțiunii curente
- CNTR (Counter): Un registru pe 14 biți dedicat buclelor (loops).
- IMAS: Gestionează întreruperile. Îți permit să ignori anumite întreruperi, să le activezi sau să forțezi declanșarea lor.

Această familie de microcontrolere mai are o limitare la ce implică cât de lungă poate fi o instrucțiune, astfel lungimea ei fixă este de 24 de biți. Acest lucru ne împiedică să scriem direct o valoare în memorie și trebuie să respectăm regula arhitecturală: orice date care merg spre memorie trebuie să plece dintr-un registru al procesorului. Din acest motiv toate scrise în memorie vor trebui mai întâi încărcate în registrul SI și după mutate în memoria de date.

Exemplu $si=0.5r$ (r – real, calculează automat valoarea hexazecimală)

$dm(ref)=si$

Automatic Gain Control (AGC)

Automatic Gain Control (AGC) urmărește și compensează variațiile de amplitudine ale unui semnal, ajustând câștigul în funcție de un nivel prescris de proiectant.

Cod

Folosim o medie mobilă a ieșirii pe M eșantioane, reducând fluctuațiile bruște.

Inițializări (start:)

Valoarea câștigului se salvează în 2 variabile pentru că DSP lucrează cu valori mai mici ca 1.0, dar amplificarea poate fi mai mare, astfel:

- $ay1=0$; -> salvează partea întreagă a câștigului
- $ax1=0$; -> salvează partea fracționară a câștigului

Referinței (amplitudinea dorită) și a pasului de adaptare:

- $dm(ref) = 0.5r$;
- $dm(mu)=0.5r$;

Setarea pentru operația de împărțire la 4 prin shiftare la dreapta cu 2 biți:

- $se=-K$;

Crearea unui buffer circular de mărime 4 la adresa delay. Aici se păstrează ultimele 4 valori ale $|y(n)|$ pentru a calcula media S

- $l3=M$;
- $i3=delay$;
- $m3=1$;

Activarea întreruperilor care declanșează rutina la fiecare eșantion nou

- $imask=0x200$;

Bucula principală de procesare (agc:)

a. Aplicarea câștigului ($y(n) = g(n-1)*x(n)$)

```
call read_input;           // citește eșantionul x(n) în registrul 'ar'
mx0=ar;                    // salvează intrarea
mr=0;                      // curăță acumulatorul (rezultatul înmulțirii)
my0=-1.0r;                 // my0 ia valoarea -1.0
ar = ax1-1;                // verifică dacă partea întreagă a lui g(n-1) este 1
if lt jump g_fr;           // dacă ax1 < 1 (adică e 0), sari peste linia următoare
mr=mr-mx0*my0(ss);         // mr = 0 - x(n)*(-1.0) = x(n). Adaugă x(n) întreg la rezultat
g_fr:
my1=ay1;                   // încarcă partea fracționară a câștigului în my1
mr=mr+mx0*my1(rnd);        // mr = mr + x(n) * partea_fracționară
dm(tx_buf+2)=mr1;          // scrie rezultatul y(n) (aflat în mr1) pe ieșire
```

b. Valoarea absolută și salvarea în linia de întârziere

```
ar = abs mr1;              // ar = |y(n)| (valoarea absolută)
sr=ashift ar (hi);         // sr1 = ar >> 2 (shiftează la dreapta cu 2 poziții). Asta înseamnă împărțire la 4 (M)!
dm(i3,m3)=sr1;             // salvează valoarea scalată în buffer-ul circular 'delay'
```

c. Calculul mediei S

```
cntr=M-1;                  // Setează contorul buclei la 3 (M-1)
mr=0, mx0=dm(i3,m3);       // mr = 0, încarcă prima valoare din buffer în mx0
do sum until ce;            // execută bucla de jos până contorul ajunge la 0
sum: mr=mr-mx0*my0(ss), mx0=dm(i3,m3); // mr = mr + mx0 (deoarece my0 e -1). Încarcă următoarea valoare.
mr=mr-mx0*my0(rnd);        // adună și ultima valoare
if mv sat mr;              // dacă apare overflow (saturare), plafonează la valoarea maximă
dm(S)=mr1;                 // Salvează rezultatul mediei S în memorie
```

d. Calculul erorii și actualizarea câștigului

```
ax0=dm(ref);               // ax0 = REF
```

```

ay0=dm(S);           // ay0 = S
ar=ax0-ay0;          // ar = REF - S (eroarea de amplitudine)
my1=ar;              // my1 = eroarea
mx1=dm(mu);          // mx1 = pasul mu (0.05)
mr=mx1*my1 (ss);     // mr1 = mu * (REF - S)

```

```

ar=mr1+ay1;          // ar = g(n-1)_fracționar + mu*(REF-S)

```

e. Gestionarea formatului numeric (Overflow-ul pe partea fracționară)

```

af = pass ar;         // Trece rezultatul noului câștig prin ALU
if ge jump cont;      // Dacă rezultatul e >= 0, totul e ok (nu a depășit granițele), sari la 'cont'
// Dacă codul ajunge aici, înseamnă că partea fracționară (ay1) a depășit 1.0
// sau a scăzut sub 0. Trebuie ajustată partea întreagă (ax1).
af = ax1-1;
if lt jump cor1;
// Cazul în care ax1 era 1, iar câștigul a scăzut sub 1.0
ax1=0;                // partea întreagă devine 0
ay1=0x7fff;           // partea fracționară devine 0.9999 (maxim pozitiv)
rti;                  // Return from Interrupt (gata)
cor1:
// Cazul în care ax1 era 0, iar câștigul a crescut peste 1.0
ax1=1;                // partea întreagă devine 1
ay1=0;                // partea fracționară reîncepe de la 0
rti;
cont:
ay1=ar;               // Dacă nu a fost overflow, doar salvează noul câștig fracționar în ay1
rti;                  // Gata

```

Adaptive line enhancer (ALE)

Codul este împărțit în 2 fișiere

1. ALE_TEST.DSP care asigură controlul și mutarea datelor
2. ALE_FCT.DSP care este "creierul" matematic

Cod

ALE_TEST.DSP

a. Inițializări registre generatoare de adrese

```

m1=1;                // Pasul de avansare în memorie este 1
m5=1;                // Pasul de avansare în program memory este 1
si=0;                // Registrul 'si' primește 0 (pentru a goli memoria)
i2=input;             // i2 indica spre începutul bufferului de intrare
l2=n;                // l2=n face ca i2 sa fie un buffer circular (hardware)
i4=input+delay;       // i4 implementează direct blocul de întârziere z^-D
l4=n;                // l4=n face ca i4 sa fie buffer circular
i3=fir_d;             // i3 indica spre linia de întârziere internă a filtrului
l3=n;                // l3=n face ca i3 să fie buffer circular
i6=hh;               // i6 indica spre coeficienții h(k)
l6=n;                // l6=n face ca i6 să fie buffer circular

```

b. Inițializare linii întârziere și coeficienți cu zero

```

cntr=n;           // Setează contorul buclei la lungimea n
do init_buf until ce; // Repeta instrucțiunile de mai jos de n ori
dm(i2,m1)=si;     // Pune 0 (din 'si') în input[n] și avansează
dm(i3,m1)=si;     // Pune 0 în fir_d[n] și avansează
init_buf:
    pm(i6,m5)=si;  // Pune 0 în coeficienții hh[n] și avansează
    imask =0x200;  // Activează întreruperea hardware
wait: jump wait;   // Bucla infinită (așteaptă pasiv întreruperile)

```

c. Rutina de servire a întreruperii de eșantionare (vine un eșantion nou)

```

input_samples:
    ena sec_reg;    // Folosește setul secundar de registre pentru a fi super rapid
    ena ar_sat;     // Activează saturarea în ALU (previne depășirile aritmetice)
    call read_input; // Citește eșantionul xz(n); rezultatul intră în 'ar'
    dm(save)=ar;    // Salvează xz(n) în variabila temporară 'save'
    dm(i2,m1)=ar;   // Baga xz(n) în linia de întârziere principală
    ar=dm(i4,m5);   // scoate eșantionul întârziat xz(n-D) folosind i4
    dm(i3,m1)=ar;   // pune eșantionul xz(n-D) în linia de întârziere a filtrului (i3)

```

d. Calculează ieșirea FIR (semnalul util y(n))

```

cntr=n-1;        // Setează contorul pentru calculele filtrului
call fir;         // Executa matematica. Ieșirea y(n) va veni în 'mr1'

```

e. Scrie ieșirea (semnalul "curățat") către convertorul digital-analogic

```

call write_output; // în mr1 se află deja valoarea filtrată y(n) gata de scriere

```

f. Calcul eroare $e(n) = xz(n) - y(n)$

```

ar=dm(save);     // Aduce intrarea inițială xz(n) din 'save' înapoi în 'ar'
ay0=ar;          // O mută în 'ay0' (intrare în sumator)
ar=ay0-mr1;      // Calculează eroarea: ar = xz(n) - y(n)
mx0=ar;          // Mută eroarea în 'mx0' pentru a fi folosită de algoritmul LLMS

```

g. Pregătire parametri pentru adaptarea coeficienților

```

cntr=n;          // Numărul de coeficienți de adaptat
my1=mu;          // Încarcă pasul de adaptare 'mu' în 'my1'
m4=-1;          // Pas negativ pentru parcurgerea inversă
m6=2;            // Pas pentru actualizare (sare 2 poziții)
m3=-1;          // Pas negativ
m7=0;           // Pas zero (nu muta pointerul după citire)
mx1=lambda;     // Încarcă factorul de scurgere 'lambda' în 'mx1'
call llms;       // Apelează "creierul" care ajustează coeficienții
dis sec_reg;     // Revine la setul primar de registre
rti;            // Gata! Așteaptă următorul eșantion

```

ALE_FCT.DSP**a. FIR Subroutine(Calculează y(n))**

```

fir:
    mr=0, mx0=dm(i3,m1), my0=pm(i6,m5); // Curăță acumulatorul (mr=0). Aduce prima dată în
mx0 și primul coeficient în my0.
    do sop until ce;           // Începe bucla hardware pentru adunare + înmulțire.
sop:

```

```
mr=mr+mx0*my0(ss), mx0=dm(i3,m1), my0=pm(i6,m5); // mr=mr+(h*x).
```

```
mr=mr+mx0*my0(rnd); // Efectuează si ultima înmulțire rotunjind rezultatul (rnd).  
if mv sat mr; // Daca acumulatorul a dat pe dinafara (overflow/mv), plafonează-l la  
maxim (saturare).  
rts; // Iese. Rezultatul final y(n) se găsește în 'mr1'.
```

b. LLMS Modify Subroutine (Adaptează h(k))

llms:

```
mr=mx0*my1(rnd), mx0=dm(i3,m1); // Calculează un termen fix: mr1 = eroare (din mx0) * mu  
(din my1). Aduce data x(n-k) in mx0.
```

```
my0=mr1; // Blochează constanta (eroare*mu) în 'my0' ca să o folosim la toți coeficienții.  
mr=mx0*my0(rnd), ay0=pm(i6,m5); // Calculează: mr1 = x(n-k) * (eroare*mu). Aduce primul  
coeficient VECHI h(k) in 'ay0'.
```

```
do adaptive1 until ce; // Repeta bucla pentru toți coeficienții
```

c. Partea de corecție a erorii (LMS normal):

```
ar=mr1+ay0, mx0=dm(i3,m1), ay0=pm(i6,m4); // ar = h(k) + [x(n-k)*mu*eroare]. Aduce în avans  
următorul 'x' (mx0) și următorul 'h' (ay0).
```

d. Partea de atenuare (Leakage):

```
mf=mx1*my1(rnd), sr1=pm(i6,m7); // Calculează mf = lambda (mx1) * mu (my1).  
mr=sr1*mf(rnd); // mr1 = h(k) * (lambda*mu)  
ay1 = mr1; // Mută partea de atenuare în 'ay1' pentru a o putea scădea  
în ALU.
```

e. Finalizarea ecuației:

```
ar=ar-ay1; // SCADE scurgerea: ar = [Corecția dinainte] - [h(k)*mu*lambda].
```

adaptive1:

```
pm(i6,m6)=ar, mr=mx0*my0(rnd); // pun noul 'ar' înapoi în memoria coeficienților! Apoi  
calculează [x * mu * eroare] pentru următorul pas din bucla.
```

```
modify(i3,m3); // Reset pointeri după ce s-a terminat toata bucla
```

```
modify(i6,m4); // Reset pointeri
```

```
rts; // Întoarcere la programul de test!
```

Median Filter (MF)

Cod

Inițializare

```
l3=W; // Setează lungimea bufferului circular la W (adică 3)  
i3=delay; // Pointerul i3 arată către începutul vectorului 'delay'  
m3=1; // Pasul de avansare prin memorie va fi 1  
l0=0; // Dezactivează circularitatea pentru pointerul i0 (va fi folosit pentru bufferul liniar)  
imask=0x200; // Activează întreruperea hardware IRQ2
```

Citirea și actualizarea istoricului (Rutina median_f:)

```
call read_input; // Citește eșantionul x(n) și îl pune în registrul 'ar'  
dm(i3,m3)=ar; // Scrie valoarea din 'ar' în memoria circulară și avansează i3 cu m3 (1 pas)
```

Copierea datelor într-un buffer temporar

```
i0=delay_sorted; // Setează i0 să arate spre noul vector unde vom copia datele
```

```

cntr=W;           // Setează contorul buclei la 3 (W)
do copy until ce; // Repetă bucla următoare până expiră contorul
si=dm(i3,m3);     // Citește o valoare din istoric (folosind pointerul circular i3) în 'si'
copy:
dm(i0,m3)=si;     // Scrie valoarea din 'si' în noul vector (folosind i0)
i2=delay_sorted; // i2 devine pointerul de bază către vectorul ce trebuie sortat
call sort_sel;    // Apelează funcția de sortare

```

Sortarea prin Selecție (Rutina sort_sel:)

a. Pregătirea buclei exterioare (loop_i)

```

m3=1;           // Ne asigurăm că pașii prin memorie se fac cu 1
l0=0; l1=0;     // Asigură lucrul liniar (nu circular) pentru pointerii i0 și i1
cntr=W;         // Contorul pleacă de la 3 (W)
do loop_i until ce; // Începe bucla mare
ax0=W;         // ax0 = 3
ay0=cntr;      // ay0 = cât a mai rămas din contor (3 la prima rulare)
ar=ax0-ay0;    // ar = 3 - 3 = 0. Acesta este indexul 'i'!
m0=ar;        // m0 = indexul minimului temporar (min = i)
m1=ar;        // m1 = indexul i

```

b. Calculul elementelor pentru bucla interioară (loop_j)

```

ar=ar+1;       // ar = i + 1
m2=ar;        // m2 va reprezenta indexul de plecare pentru j
ay0=m1;       // ay0 = i
ar=ax0-ay0;    // ar = W - i (ex: 3 - 0 = 3)
ar=ar-1;      // ar = W - i - 1. Reprezintă de câte ori se va executa bucla j
if eq jump loop_i; // Dacă numărul de execuții e 0, sari la finalul buclei mari

```

```

cntr=ar;       // Setează contorul buclei mici la valoarea calculată
i0=i2;        // Pointerul i0 arată din nou la începutul vectorului A
modify(i0,m2); // Mută pointerul direct la elementul A[i+1]

```

c. Căutarea minimului

```

do loop_j until ce; // Repetă bucla mica de verificare
ax0=dm(i0,m3);     // ax0 citește A[j] (elementul curent de verificat)
i1=i2;            // i1 se întoarce la începutul vectorului A
modify(i1,m0);    // i1 sare la indexul minimului curent
ay0=dm(i1,m3);    // ay0 citește A[min]
ar=ax0-ay0;      // Compară: A[j] - A[min]
if gt jump loop_j; // Dacă rezultatul e pozitiv (A[j] e mai MARE), sari peste și continuă bucla

```

Dacă nu am sărit, înseamnă că am găsit un număr mai mic! Așa că reținem noul index:

```

ax0=W;         // Re-calculăm valoarea lui j din contoare
ay0=cntr;
ar=ax0-ay0;    // ar = j
m0=ar;        // Actualizăm m0: min = j
loop_j: nop;   // Finalul buclei mici

```

d. Interschimbarea (Swap)

```

i1=i2; modify(i1,m1); // Mută i1 pe poziția i
ax1=dm(i1,m3);       // ax1 = A[i] (variabila temporară 'tmp')
i1=i2; modify(i1,m0); // Mută i1 pe poziția minimului găsit

```

```

ay1=dm(i1,m3);      // ay1 = A[min]
i1=i2; modify(i1,m1);
dm(i1,m3)=ay1;      // Scrie valoarea minimă la poziția i: A[i] = A[min]
i1=i2; modify(i1,m0);
dm(i1,m3)=ax1;      // Scrie valoarea lui A[i] (tmp) la fosta poziție a minimului

```

Extragerea mediane și ieșirea

```

i0=delay_sorted;    // Se mută la începutul vectorului abia sortat
m0=K;               // Setează m0 la constanta K (care e 1)
modify(i0,m0);      //Modifică adresa pointerului i0: sare cu K poziții peste capul vectorului (fix
la mijloc)
ar=dm(i0,m3);       // Citește elementul median în registrul 'ar'
dm(tx_buf+2)=ar;    // Trimite rezultatul median către ieșirea sistemului
rti;               // Returnează din întrerupere, așteaptă următorul eșantion audio!

```

Rezultate

Pentru a face posibilă afișarea semnalelor noastre de intrare și ieșire a trebuit să facem următoarele stări în aplicație:

1. Am adăugat proiectul și i-am dat build
2. L-am rulat odată pentru a vedea la ce adrese sunt rx_buf și tx_buf
3. Am creat streamuri pentru fișierele de intrare și ieșire
4. Am creat întreruperea pe IRQ2, care este aceeași pentru toate 3
5. Am pus în memoria de date fișierul de intrare și am dat run apoi l-am adăugat pe cel de ieșire
6. Am afișat de la adresa din memorie semnalul de intrare și ieșire

Automatic gain control

a. Adresele rx_buf și tx_buf

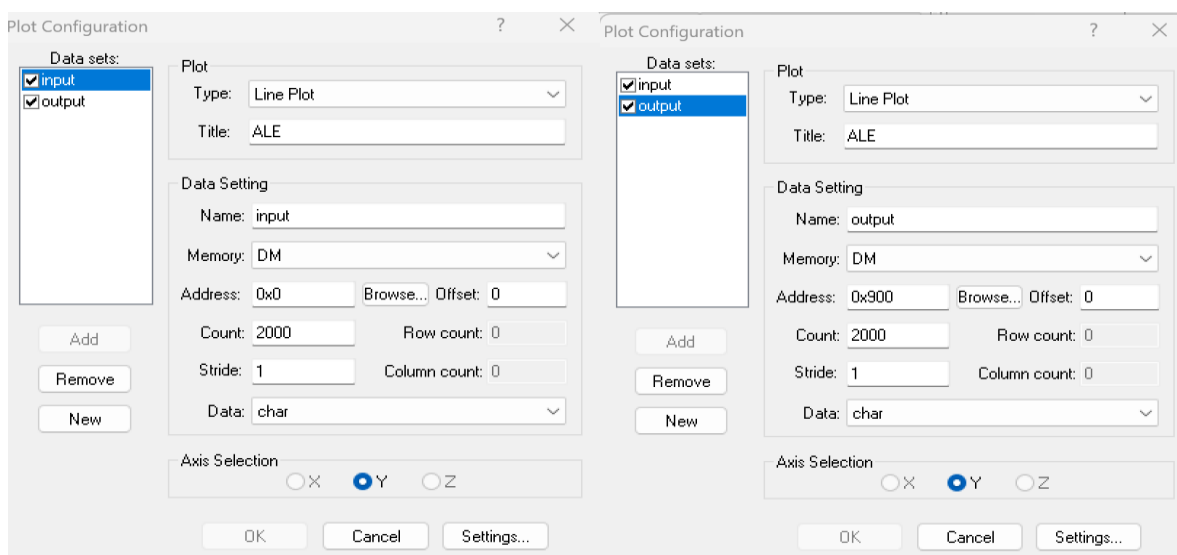
```

delay
[000000] 0FFB 0FFB 0FFB 0FFB
tx_buf
[000004] 0000 0000 3FEE
S
[000007] 3FEC
rx_buf
[000008] 0000 0000 6000
ref
[00000B] 4000
mu
[00000C] 0666 uuuu uuuu uuuu u

```

Memoria de date

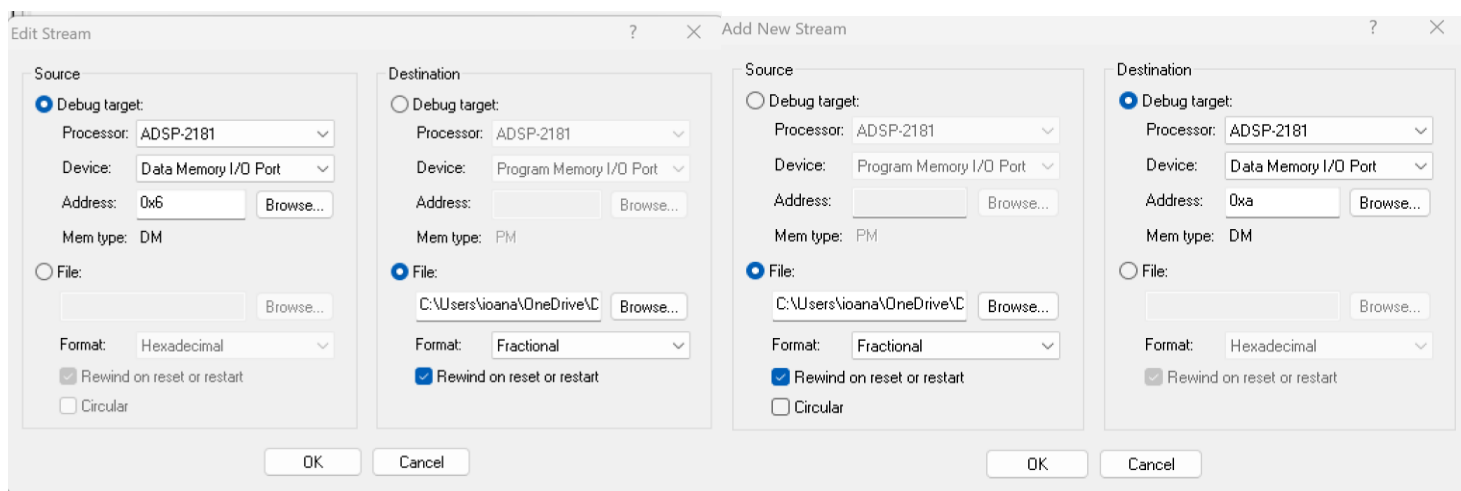
b. Streamurile pentru fişierele de intrare şi ieşire



Plot intrare

Plot ieşire

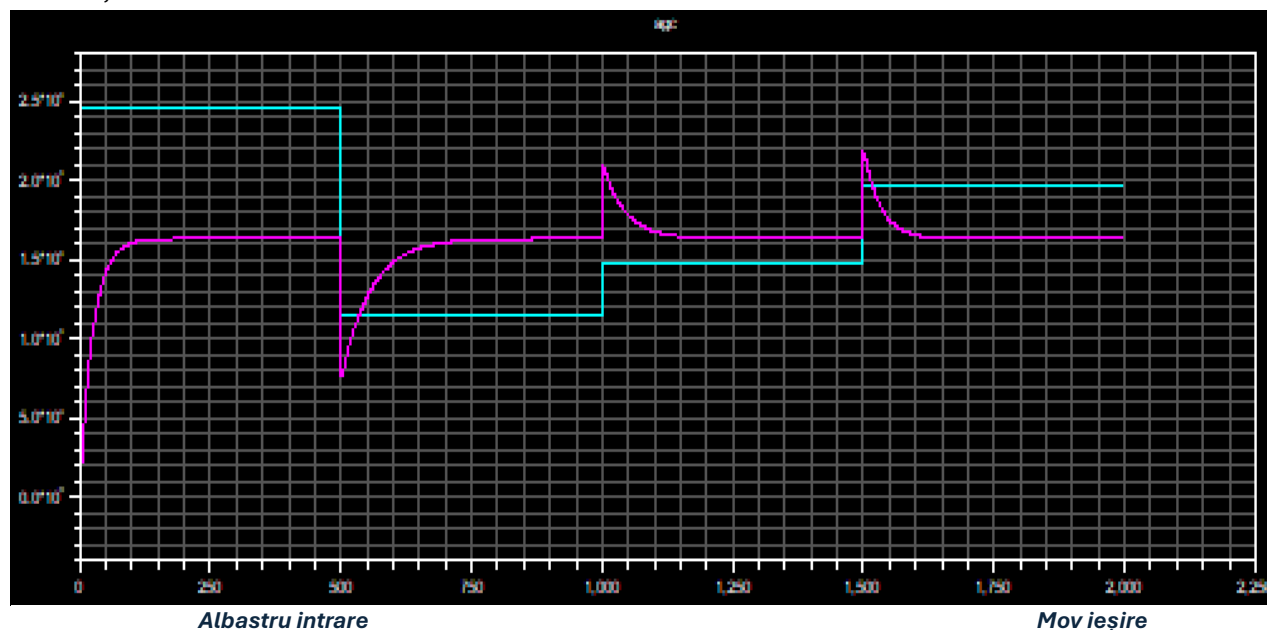
c. Locaţia fişierului de intrare şi a celui de ieşire



Stream de ieşire

Stream de intrare

d. Afişarea semnalelor



Adaptive line enhancer

a. Adresele rx_buf și tx_buf

```
tx_buf
[000200] 0000 0000 C1C6
save
[000203] C194
rx_buf
[000204] 0000 0000 C194
```

Memoria de date

b. Streamurile pentru fișierele de intrare și ieșire

Stream de ieșire

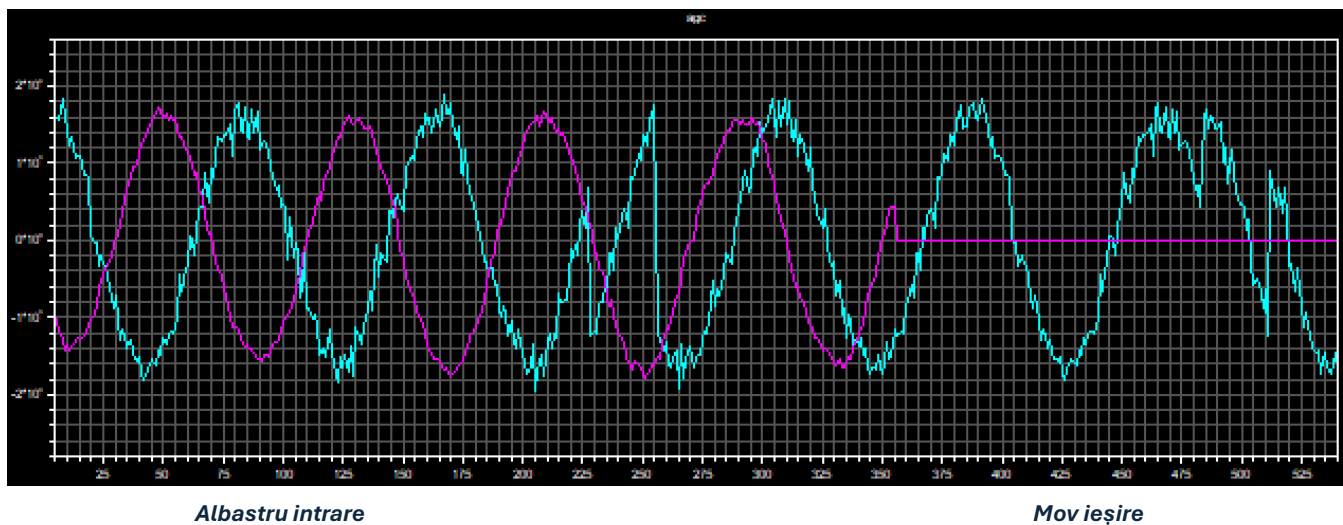
Stream de intrare

c. Locația fișierului de intrare și a celui de ieșire

Plot intrare

Plot ieșire

d. Afișarea semnalelor



Median Filter

a. Adresele rx_buf și tx_buf

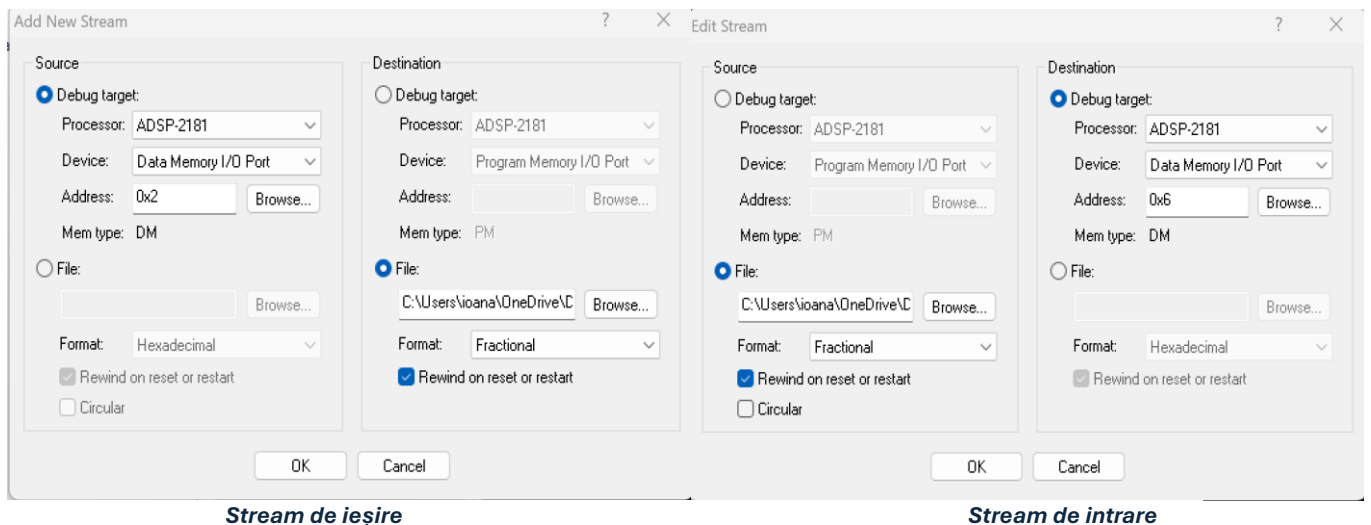
```

tx_buf
[000000] 0000 0000 4889 0000
rx_buf
[000004] 0000 0000 uuuu 0000
delay
[000008] 3EB9 521C 4889
delay_sorted
[00000B] 3EB9 4889 521C uuuu

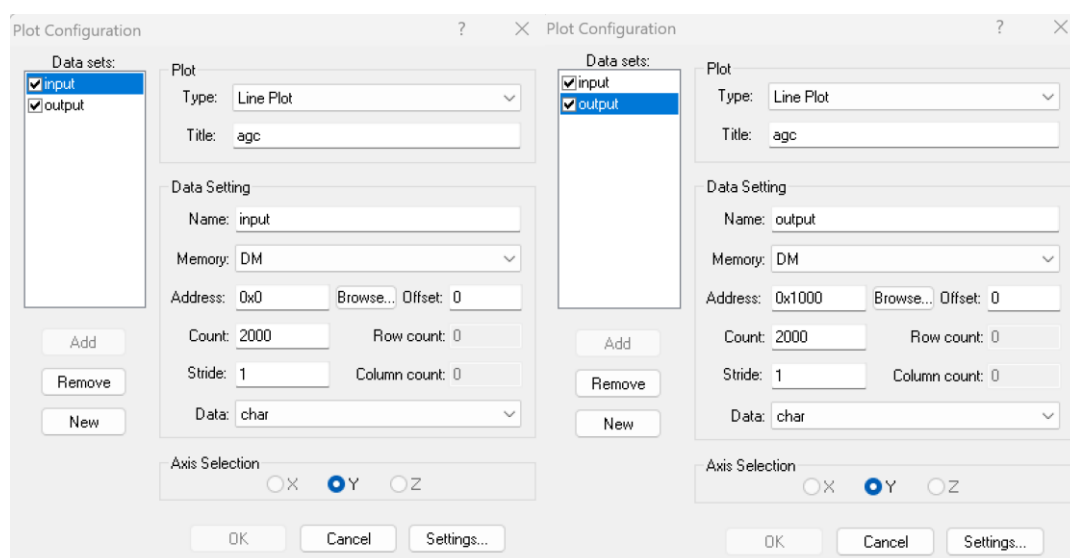
```

Memoria de date

b. Streamurile pentru fișierele de intrare și ieșire



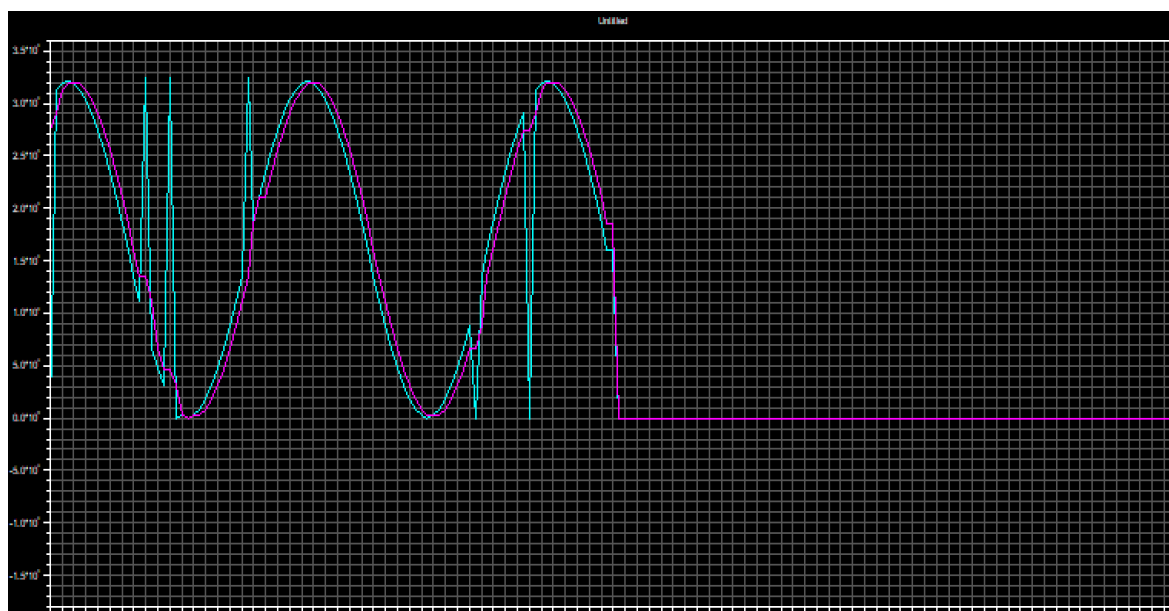
c. Locația fișierului de intrare și a celui de ieșire



Plot intrare

Plot ieșire

d. Afișarea semnalelor



Albastru intrare

Mov ieșire

III. Asamblarea funcțiilor într-un cod unic

Definirea parametrilor

Parametri Filtre

```
#define M_AGC 4           // Dimensiunea ferestrei pentru media mobila (S) din AGC
#define K_AGC 2           // Parametru pentru shiftare dreapta:  $2^2 = 4$  (împărțire la 4)
#define W_MF 3            // Dimensiunea ferestrei filtrului Median (3 eșantioane)
#define K_MF 1            // Indexul valorii mediane într-un vector sortat de 3 elemente (poziția 1)
#define N_ALE 256         // Numărul de coeficienți ai filtrului adaptiv FIR
#define D_ALE 128         // Valoarea întârzierii de decorelare ( $z^{-D}$ )
```

Buffere DM

```
.SECTION/DM buf_var1; // Secțiune de memorie de date pentru bufferul de recepție
.var rx_buf[3];        // Buffer circular recepție CODEC: Status, Canal Stânga, Canal Dreapta
.SECTION/DM buf_var2; // Secțiune de memorie de date pentru bufferul de transmisie
.var tx_buf[3] = 0xc000, 0x0000, 0x0000; // Buffer circular transmisie către CODEC (inițializat cu
                                         // un status valid)
.SECTION/DM buf_var3; // Secțiune memorie pentru comenzile de inițializare ale CODEC-ului
                     AD1847
.var init_cmds[13] = 0xc002, 0xc102, 0xc288, 0xc388, 0xc488, 0xc588, 0xc680, 0xc780, 0xc85c,
0xc909, 0xca00, 0xcc40, 0xcd00; // Secvența de configurare hardware CODEC
.SECTION/DM data1;     // Secțiune pentru variabile generale în memoria de date (DM)
```

Parametri de control

```
.var stat_flag;        // Flag de stare pentru transmiterea comenzilor inițiale
.var PF_input;         // Variabilă în care se memorează citirea portului Programmable Flags
.var id_dsp;           // Retine ID-ul extras din comandă (baza D2,D1,D0)
.var cda;              // Retine algoritmul selectat curent (0=AGC, 1=ALE, 2=MF)
.var canal;           // Retine canalul pe care se face procesarea (Stânga/Dreapta)
.var temp_out;         // Variabilă temporară pentru scrierea la ieșire
.var temp_in;          // Variabilă temporară pentru salvarea eșantionului de intrare x(n)
.var cda_prev;         // Salvează comanda executată anterior (pentru a detecta schimbările de
                     // comandă)
.var Flag_CDA;         // Flag care semnalizează apariția întreruperii asincrone ce determină citirea
                     // comenzii
```

Variabile AGC

```
.var/circ delay_agc[M_AGC]; // Linie de întârziere (buffer circular) pt salvarea ultimelor 4 valori |y(n)|
.var S_agc;                 // Retine suma modulelor împărțite la 4 ale eșantioanelor de ieșire
.var ref_agc;               // Nivelul de Referință (R) la care trebuie să ajungă semnalul
.var mu_agc;                // Pasul de adaptare care reglează viteza buclei AGC
.var agc_flag_init;         // Semnalizează dacă AGC-ul are nevoie să-și reseteze constantele interne
                     // prin parcurgerea inițializării
.var g_int_agc;             // Câștigul curent g(n-1) - partea întreagă (stocat separat pt a permite câștig > 1)
.var g_fr_agc;              // Câștigul curent g(n-1) - partea fracționară
```

Variabile MF

```
.var/circ delay_mf[W_MF];    // Buffer circular MF (3 eşantioane brute x(n))
.var delay_sorted[W_MF];    // Buffer liniar temporar în care se ordonează (sortează) valorile
.var mf_flag_init;          // Flag de inițializare MF
```

Variabile ALE

```
.var/circ fir_d[N_ALE];      // Linia de întârziere a filtrului adaptiv FIR (xz(n-D))
.var/circ input_ale[N_ALE];  // Linia de întârziere principală a semnalului (pt blocul delay z-D)
.var mu_ale;                 // Pasul de adaptare pt filtrul adaptiv
.var lambda_ale;             // Factorul Leakage pt a preveni divergentele coeficienților
.var ale_flag_init;          // Flag de inițializare ALE
.SECTION/PM pm_da;           // Secțiune de variabile în memoria de program (PM) - pt paralelizare
```

Variabile PM ALE

```
.var/circ hh[N_ALE];        // Coeficienții filtrului adaptiv FIR (h(k)) plasați în PM pt a fi citați simultan cu
                             // datele din DM
```

Tabela vectori întreruperi

În ADSP-2181, Tabela Vectorilor de Întrerupere (Interrupt Vector Table) se află fizic chiar la începutul Memoriei de Program (începând cu adresa 0x0000). Hardware-ul alocă strict 4 instrucțiuni (4 locații de memorie) pentru fiecare întrerupere în parte.

rti (Return from Interrupt) = readuce programul în punctul din cod în care s-a declanșat întreruperea.

Pentru întreruperile nefolosite din tabela de întreruperi, spațiul locațiilor de memorie conține doar instrucțiuni rti pentru ca orice întrerupere declanșată accidental să determine revenirea la fragmentul de cod unde a primit întreruperea.

```
.SECTION/PM interrupts;     // Locațiile hardware fixe unde procesorul sare la diferite evenimente
    jump start;    rti; rti; rti;    /*00: reset - După pornire, sari la rutina 'start'*/
    ax0=1; dm(Flag_CDA)=ax0; rti; rti; /*04: IRQ2: întrerupere asincronă */
                                   /* scriere în două instrucțiuni pentru că scrierea în memorie
                                   trebuie să treacă printr-un registru */
    rti;           rti; rti; rti;    /*08: IRQ1 */
    rti;           rti; rti; rti;    /*0c: IRQ0 */
    ar = dm(stat_flag);              /*10: SPORT0 tx - Aici hardware-ul sare automat de fiecare dată
                                   când DSP-ul termină de trimis un pachet de date către CODEC.
    ar = pass ar;                    // Testează flagul de status
    if eq rti;                       // Dacă e 0, revine (nu mai e nimic de inițiat)
    jump next_cmd;                   // Dacă e 1, sari să trimiți următoarea comandă de inițializare a CODEC-ului
    jump input_samples;              /*14: SPORT0 rx (recepție SPORT0)*/
    rti; rti; rti;
    rti;           rti; rti; rti;    /*18: IRQE */
    rti;           rti; rti; rti;    /*1c: BDMA */
    rti;           rti; rti; rti;    /*20: SPORT1 tx or IRQ1 */
    rti;           rti; rti; rti;    /*24: SPORT1 rx or IRQ0 */
    nop;           rti; rti; rti;    /*28: timer */
    rti;           rti; rti; rti;    /*2c: power down */
.SECTION/PM seg_code;         // Secțiunea de cod propriu-zis
```

Inițializare sistem și filtre

start:

```
ax0 = b#0000100000000000; // Configurare registru de control sistem
dm (Sys_Ctrl_Reg) = ax0; // Scrie valoarea
ena timer; // Activează timer-ul hardware
```

Inițializare pointeri buffere circulare RX/TX/comenzi */

```
i5 = rx_buf; l5 = LENGTH(rx_buf); // Setare index(i5) si lungime(l5) pentru bufferul circular de
RECEPTIE SPORT0
i6 = tx_buf; l6 = LENGTH(tx_buf); // Setare index(i6) si lungime(l6) pentru bufferul circular de
TRANSMISIE SPORT0
i3 = init_cmds; l3 = LENGTH(init_cmds); // Setare pointeri pentru tabloul cu comenzi inițiale
m1 = 1; m5 = 1; // Setează regiștrii Modify cu valoarea 1 (incrementare cu 1 la citire/scriere)
```

Configurare SPORT0

```
ax0 = b#0000110011010111; dm (Sport0_Autobuf_Ctrl) = ax0; // Config autobuffering, activat rx/tx
ax0 = 0; dm (Sport0_Rfsdiv) = ax0; // Factor divizare RFS
ax0 = 0; dm (Sport0_Scldiv) = ax0; // Factor divizare ceas serial SCLK
ax0 = b#1000011000001111; dm (Sport0_Ctrl_Reg) = ax0; // Cuvânt de control 16-bit, I2S
compatibil
ax0 = b#00000000000000111; dm (Sport0_Tx_Words0) = ax0; // Config canale transmitere
ax0 = b#00000000000000111; dm (Sport0_Tx_Words1) = ax0;
ax0 = b#00000000000000111; dm (Sport0_Rx_Words0) = ax0; // Config canale recepție
ax0 = b#00000000000000111; dm (Sport0_Rx_Words1) = ax0;
```

Configurare sistem

```
ax0 = b#0001100000000000; dm (Sys_Ctrl_Reg) = ax0; // Finalizare config sistem
ifc = b#00000011111110; // Clear la orice întreruperi vechi (curata flagurile)
nop;
icntl = b#00010; // Setare priorități întreruperi
mstat = b#1100000; // Configurare registru stare mașina (MSTAT)
jump skip; // Sari la continuarea inițializării
```

Inițializare codec ad1847

```
ax0 = 1; dm(stat_flag) = ax0; ena ints; imask = b#0001000001; // Setează flag transmitere comenzi;
activează temporar întreruperile
ax0 = dm (i6, m5); tx0 = ax0; // Pune prima comanda in registrul de transmisie
check_init:
ax0 = dm (stat_flag); af = pass ax0; if ne jump check_init; // Așteaptă pana stat_flag devine 0
(pana s-au trimis toate cele 13 comenzi)
ay0 = 2;
check_aci1:
ax0 = dm (rx_buf); ar = ax0 and ay0; if eq jump check_aci1; // Rutine de test auto-calibrare
CODEC (așteaptă bit validare)
```

DSP-ul stă și ascultă pe recepție (rx_buf), până la momentul în care CODEC-ul termină auto-calibrarea și schimbă stat_flag înapoi în 0.

```
check_aci2:
```



```
ax0 = dm (rx_buf); ar = ax0 and ay0; if ne jump check_aci2; idle; // Așteaptă terminarea calibrării
```

Maschează MSB-urile cuvintelor de câștig înainte de scriere in codec

```
ay0 = 0xbf3f; ax0 = dm (init_cmds + 6); ar = ax0 AND ay0; dm (tx_buf) = ar; idle; // Ajustări finale filtre  
anti-aliasing CODEC
```

```
ax0 = dm (init_cmds + 7); ar = ax0 AND ay0; dm (tx_buf) = ar; idle;
```

Reactivează întreruperea SPORT0 RX după init codec

```
ifc = b#000000011111110; nop; imask = b#0001100001; // Curata iar flagurile după init
```

```
skip:
```

```
imask = 0x220; // 0000 0010 0010 0000 -> activam IRQ2 (pentru schimbare comanda) si  
SPORT0 rx (pentru prelucrare eşantion cu eşantion)
```

```
si=0xFFFF; dm(Dm_Wait_Reg)=si; // Configurează Wait States pt memoria externa
```

Setare PF ports exclusiv ca INTRARI

```
si=0x0000; dm(Prog_Flag_Comp_Sel_Ctrl)=si; // Toți cei 8 pini Prog_Flag (PF0-PF7) devin INTRARI  
dinspre procesorul ARM
```

```
si=3;
```

```
dm(cda_prev)=si; // Inițializează variabila algoritmului anterior cu 3 (valoare pentru a forta  
execuția)
```

```
si=1;
```

```
dm(Flag_CDA)=si; // sistemul să citească măcar o dată starea butoanelor fizice pentru a ști cu  
ce filtru să înceapă după reset
```

```
wt:
```

```
nop;
```

```
jump wt; // Bucla infinita (Main program loop). Iese de aici doar când SPORT0 generează  
întrerupere
```

ISR SPORT0: Citire și rutare comanda

Pentru izolarea logicii de comanda de logica de prelucrare, ne-am gândit la următoarele optimizări:

Conform organigramei programului, întreruperea asincronă cu rolul de a determina citirea comenzii va fi reprezentată de întreruperea IRQ2. Așadar, am introdus variabila Flag_CDA care să fie setată de apariția întreruperii IRQ2 și să determine, la momentul primirii unui nou eşantion, citirea comenzii de pe PF, prelucrarea separată a câmpurilor de biți și apelarea rutinei corespunzătoare, apoi resetarea Flagului pentru ca la următoarele eşantioane comanda să se considere deja cunoscută.

Întreruperea care dictează preluarea datelor de intrare și prelucrarea lor este SPORT0 rx, apelând la fiecare nou eşantion rutina input_samples.

```
input_samples:
```

```
ena_sec_reg; // Activează bancul secundar de registre; salvează starea curenta (Main Loop)  
a registrelor ALU/MAC pt viteza
```

Tratarea întreruperii asincrone

```
ax0 = dm(Flag_CDA); // Citim stegulețul
```

```
ar = pass ax0; // Trecem prin ALU (necesar pentru a actualiza flagurile lui ALU si a face  
jumpuri condiționate)
```

```
if eq jump procesare; // Daca e 0, NU s-a apăsat butonul.
```

Citirea portului PF

```
ax0=dm(Prog_Flag_Data); // Citește portul hardware fizic - aici e valoarea trimisa de ARM
```

(Comanda 8-biti)

```
ay0=0x00FF;  
ar=ax0 and ay0;    // Maschează biții inutili  
dm(PF_input)=ar;    // Salvează comanda brută citită în memorie
```

Parsare comanda din PF_input

```
ax0 = dm(PF_input); // Pune comanda in ALU
```

Validare: D7,D6 trebuie sa fie "11"

```
ay0 = 0x00C0;  
ar = ax0 AND ay0;    // Izolează D7,D6 (masca 1100 0000)  
ay1 = 0x00C0;    // valoarea hard codată a biților D7, D6 valizi  
ar = ar - ay1;    //verifica daca amândoi sunt '1'  
if ne jump invalid_cmd; // Daca diferă de 11, sari la bypass (comanda invalida)
```

ID DSP: biti D2:D0 trebuie sa fie "001"

```
ay0 = 0x0007;  
ar = ax0 AND ay0;    // Izolează ultimii 3 biți  
dm(id_dsp) = ar;  
ay1 = 0x0001;  
ar = ar - ay1;    // verifica daca ID == 1  
if ne jump invalid_cmd; // Daca alt DSP e adresat, acest DSP da bypass
```

Canal

```
ay0 = 0x0008;    // Izolează bitul 3  
ar = ax0 AND ay0;  
se = -3;    // shift exponent negativ => shiftare dreapta  
sr = lshift ar (lo); //shiftează D3 spre D0  
dm(canal) = sr0; // salvează "canal" (0=stg, 1=dr)
```

Funcție CDA

```
ay0 = 0x0030; ar = ax0 AND ay0; se = -4; sr = lshift ar (lo); dm(cda) = sr0;  
// Izolează algoritmul (D5, D4), adu-l la baza și salvează "cda" (00, 01, 10)
```

Resetarea Flag

```
ax0 = 0;  
dm(Flag_CDA) = ax0; // RESETAM FLAG-UL! (Flag_CDA = 0)
```

Citește eșantionul de intrare x(n) */

```
call read_buff;    // Funcție helper care extrage x(n) din rx_buf in fct de 'canal'. Pune rezultatul  
in 'ar'  
dm(temp_in)=ar;    // Salvează x(n) pt folosire ulterioara in 'temp_in'
```

Decizie algoritm

```
ax0=dm(cda_prev); // Încarcă algoritmul rulat la pasul T-1  
ay0=dm(cda);    // Încarcă algoritmul curent  
ar=ax0-ay0;    // Scade  
if eq jump cont; // Daca dif==0, algoritmul a rămas același, sari peste etapa de reset inițializare
```

```

si=1; // Daca s-a schimbat, încarcă valoarea 1 in Shifter Input
dm(agc_flag_init)=si; // Setează Flag necesitate inițializare AGC pe 1
dm(mf_flag_init)=si; // Setează Flag necesitate inițializare MF pe 1
dm(ale_flag_init)=si; // Setează Flag necesitate inițializare ALE pe 1
si=dm(cda); // Actualizează comanda
dm(cda_prev)=si; // Noua comanda anterioara devine comanda curenta

```

Rutarea propriu-zisa

```

cont:
    ax0 = dm(cda); // Încarcă id algoritm curent
    ay0 = 0; ar = ax0 - ay0; if eq jump run_agc; // Daca e 00: sari la algoritmul AGC
    ay0 = 1; ar = ax0 - ay0; if eq jump run_ale; // Daca e 01: sari la algoritmul ALE
    ay0 = 2; ar = ax0 - ay0; if eq jump run_mf; // Daca e 10: sari la algoritmul MF
invalid_cmd:
    call read_buff; // Daca e o eroare de comanda, citește intrarea
    jump write_out; // Si trimite-o direct la ieșire (Bypass/Fără filtrare out=in)

```

ALGORITMUL AGC (Automatic Gain Control)

Faptul că gama dinamică este $[-1, 1]$ înseamnă că hardware-ul procesorului (unitățile sale matematice) este construit să interpreteze orice număr pe 16 biți nu ca pe un număr întreg (ex: 500, 10000), ci ca pe o fracție dintr-o valoare maximă posibilă. Acest format poartă numele de virgulă fixă fracționară (adesea formatul Q15 pentru 16 biți).

Limita inferioară este exact -1.0 (reprezentată binar ca 1000 0000 0000 0000).

Limita superioară este 0.999969... (reprezentată binar ca 0111 1111 1111 1111). Nu ajunge niciodată la 1.0 perfect curat din cauza modului în care funcționează complementul față de motivul pentru care paranteza este rotundă la dreapta.

run_agc:

Verificam daca AGC trebuie initializat

```

ar=dm(agc_flag_init); // Citeste flagul de init
ar=ar-1; // Compara cu 1
if eq call agc_init; // Daca flagul a fost 1, curata memoria, reinitializeaza referintele si bufferul circular

```

*Încarcă câștigul curent: ax1=parte intreaga, ay1=parte fractionara */*

```

ax1=dm(g_int_agc); // ax1 <- Partea intreaga a castigului g(n-1)
ay1=dm(g_fr_agc); // ay1 <- Partea fractionara a castigului g(n-1)
se=-K_AGC; // Seteaza shiftarea la dreapta cu 2 (pt impartirea la 4 in calculul mediei S)
mx0=dm(temp_in); // Aducem in MAC intrarea bruta x(n)
mr=0; // Curata acumulatorul pe 40-biti
my0=-1.0r; // Constant multiplier = -1.0

```

Verificam daca g(n-1) este >= 1.0

```

ar = ax1-1; // ar = partea intreaga - 1
if lt jump g_fr; // Daca partea intreaga e 0, sari exclusiv la inmultirea fractionara
mr=mr-mx0*my0(ss); // Daca ax1 e >= 1, acumuleaza: mr = 0 - [x(n)*(-1)] = x(n). S-a adunat 1 * x(n)
(ss) <-> modificador hardware: inmultire signed-signed (numere in compl fata de 2)
pentru ca sunt si esantioane negative

```

```

g_fr:
my1=ay1;           // Incarca partea fractionara
mr=mr+mx0*my1 (rnd);
// MAC:  $y(n) = x(n)*1$  (daca exista) +  $x(n)*\text{partea fractionara}$ . Rezultatul final rotunjit sta in mr1 (rnd)
<-> acumulatorul mr pastreaza rezultatul util in cuvantul sau superior mr1 si rotunjeste rezultatul in sus
(aduna 1) daca bitul cel mai semnificativ al lui mr0 este 1
ar=mr1;           // Copiem semnalul final in 'ar'
call write_out;    // Si il expediem catre CODEC (y(n) a fost transmis)

```

Actualizam memoria pentru calculul mediei modulelor $S(n)$

```

ar = abs mr1;       // ALU: calculeaza valoarea absoluta |y(n)|
sr=ashift ar (hi);   // Shifter: shifteaza la dreapta cu 2 pozitii (imparte cu M_AGC = 4) (se asaza
                     // datele in cuvantul cel mai semnificativ din sr pentru a se asigura ca numarul
                     // nu se pierde prin shiftare)
dm(i3,m3)=sr1;      // Scrie valoarea impartita in istoric delay_agc, apoi avanseaza circular i3

```

Calculul mediei S (suma celor 4 elemente salvate)

```

cntr=M_AGC-1;       // Bucla aduna 4 elemente. Seteaza counter = 3, la fiecare executie a
                     // liniei sum se decrementeaza
mr=0, mx0=dm(i3,m3); // MAC: goleste mr; adu in mx0 primul element
do sum until ce;     // Incepe bucla hardware (cu fiecare instructiune, counter expired = counter
                     // se decrementeaza pana ajunge 0)
sum: mr=mr-mx0*my0(ss), mx0=dm(i3,m3);
                     //  $mr = mr - (A_k * -1)$  adica  $mr = mr + A_k$ . Aduce in mx0 urmatorul element
mr=mr-mx0*my0(rnd); // Aduna si ultimul element rotunjind rezultatul in mr1
if mv sat mr;        // Protejeaza la overflow prin saturare (scrie valoarea maxim pozitiva in mr1
                     // daca in mr2 am avut overflow) (sat)
dm(S_agc)=mr1;       // Salveaza media in variabila S_agc

```

Media prin adunarea esantioanelor deja impartite la 4 ne ajuta sa evitam overflow si saturare prin incarcarea unei sume mari in mr, sacrificand 2 biti de rezolutie prin shiftare la dreapta

Calcul Eroare ($e(n) = \text{Ref} - S_agc$)

```

ax0=dm(ref_agc);    // ax0 = Referinta ideala (ex. 0.5)
ay0=dm(S_agc);       // ay0 = Media calculata
ar=ax0-ay0;          // ar = R - S (aceasta e eroarea)

```

Calcul Corectie ($\mu * \text{Eroare}$)

```

my1=ar;              // my1 = eroare
mx1=dm(mu_agc);      // mx1 = pasul de invatare mu

mr=mx1*my1 (ss);     // mr1 =  $\mu * (R - S)$ 
ar=mr1+ay1;          // Adunam corectia obtinuta (mr1) la castig FRACTIONAR vechi (ay1)
af = pass ar;         // Trecem rezultatul prin ALU pt a actualiza flag-urile
if ge jump cont_agc;  // Daca rezultatul calculat este corect si se incadreaza intre [0, 0.999], sari
                     // mai departe

```

Corectie overflow castig: limitează g între 0 și 1 */

```

// Overflow handling (Daca noul castig fractionar a devenit negativ sau a depasit limita max)

```

```
// Trebuie sa compensam/ajustam bitul de Parte Intreaga (ax1)
af = ax1-1;           // Verificam cat era partea intreaga.
if lt jump cor1;      // Daca ax1 era 0, e clar ca fractia a depasit in sus (s-a marit -> cor1).
```

```
// Cazul A: ax1 era 1, iar ay1 a scazut sub 0
ax1=0;               // Pica la ax1=0
ay1=0x7fff;          // Fractionarul sare la MAX pozitiv (0.9999). 1.0 -> 0.99
jump save_gain;
```

```
cor1:
// Cazul B: ax1 era 0, iar ay1 a sarit peste 0x7FFF (adica a depasit valoarea 1.0 totala)
ax1=1;               // Urca partea intreaga la 1
ay1=0;               // Reseteaza fractionarul la 0
jump save_gain;
```

```
cont_agc:
ay1=ar;              // Totul e in regula, inregistram rezultatul noului fractionar in ay1
```

```
save_gain:
dm(g_fr_agc)=ay1;    // Salveaza fractionarul in memorie (ptr iteratia n+1)
dm(g_int_agc)=ax1;    // Salveaza integer in memorie (ptr iteratia n+1)
rti;                 // Iese din intreruperea principala SPORT0 si revine in Main Loop
```

ALGORITMUL ALE (*Adaptive Line Enhancer*)

```
run_ale:
    ena ar_sat;       // Activează protecția ALU de tip saturate
    ar=dm(ale_flag_init); //verificam daca trebuie initializare
    ar=ar-1;
    if eq call ale_init;

    ar=dm(temp_in); //readucem in ar esantionul curent
    dm(i2,m1)=ar; // Plasează eşantionul nealterat x(n) (din 'ar') in capul liniei de întârziere
                    // principale (input_ale)
    ar=dm(i4,m5);     // Pointerul i4 indica spre x(n-D) din aceeaşi memorie. Extrage eşantionul
                    // decorelat
    dm(i3,m1)=ar;     // Plaseaza x(n-D) proaspăt in linia de întârziere a filtrului propriu-zis (fir_d)
                    // Un pointer dintr-un cartier (DAG1 sau DAG2) poate folosi orice modificador
                    // din acelaşi cartier
```

Calculeaza lesirea FIR y(n)

```
cntr=N_ALE-1;        // Filtru cu 256 coeficienti, initializam bucla ptr N-1 iterații
call fir;             // Intra in subrutina hardware FIR (y(n) rezultat in mr1)
```

Trimite semnalul catre DAC

```
ar=mr1;              //pentru a aduce valoarea rezultata in write_out
call write_out;      // mr1 (valoarea 'curata') este pregatita, o trimitem afara spre transmitere
```

Calculează eroarea $e(n) = x(n) - y(n)$

```
ar=dm(temp_in); // Extragem x(n)-ul brut, contaminat cu zgomot
ay0=ar;          // Il ducem in sursa operatiei (ay0)
ar=ay0-mr1;      // Calcul in ALU: Eroare = Semnal Brut - Semnal Prezis
mx0=ar;          // Eroarea se depoziteaza in mx0. Algoritmul Leaky LMS o are gata de multiplicare.
```

Inițializează pașii de update pentru toți N coeficienți

```
cntr=N_ALE;      // Bucla va trece prin 256 coeficienti
my1=dm(mu_ale);  // Pasul de invatare adaptiva 'mu' (viteza)
m4=-1; m6=2; m3=-1; // Configurare vectori pas (Modify registers) necesari iterarii inverse
                    //Formula actualizarii coeficientilor presupune intoarcerea cu -1
                    //pentru esantionul precedent, coeficientul urmator este pus in ay0, deci
                    //pointerul va fi cu 2 pozitii decrementat pentru parcurgerea celor 2 instr
m7=0;            // Citire fara incrementare pointer
mx1=dm(lambda_ale); // Valoarea leakage (lambda) responsabila cu stabilitatea
                    //coeficientilor in zgomot
call llms;       // Executa adaptarea Leaky LMS ce va scrie coeficientii h(k) in PM.
dis sec_reg;     // Totul a fost calculat, putem dezactiva setul paralel de registre si sa dam
                    //control inapoi sistemului
rti;             // Return din ISR
```

FIR Subroutine

Funcție standard DSP MAC pentru Filtru Liniar: $y = \sum(x \cdot h)$

```
/* i3 = ^ pointer la istoric date
i6 = ^ pointer coeficienti (in Program Memory pentru fetch dual)
mr1 = va stoca rezultatul */
fir: mr=0, mx0=dm(i3,m1), my0=pm(i6,m5); // Curata MAC, fetch dual: încarcă esantion (dm) in mx0
                                         // si coeficient(pm) in my0 simultan
do sop until ce; // Repeta hardware pana contorul = 0
sop: mr=mr+mx0*my0(ss), mx0=dm(i3,m1), my0=pm(i6,m5); // Sum of Prod: convolutie discreta
                                         // intre eşantioanele întârziate si coeficienții si pre-incarca in pipeline p-
                                         // următorul
mr=mr+mx0*my0(rnd); // Realizeaza înmulțirea reziduala pt ultimul element adunand la final
                    // si aplicând o rotunjire (rnd)
if mv sat mr;      // Daca a fost depășire capacitiva a MAC (MAC Overflow), impune saturare
rts;               // Întoarcere din subrutina
```

LLMS Modify Subroutine

// Functie Adaptiva Leaky Least Mean Squares:

// Formula: $h(k) = h_vechi(k) \cdot (1 - \mu \cdot \lambda) + \mu \cdot x(n-k) \cdot e(n)$ [cite: 3, 4]

llms:

```
mr=mx0*my1(rnd), mx0=dm(i3,m1); // mr1 = eroare (din mx0) * mu (my1) | Aduce x(n-k) din
                                         // mem in mx0
my0=mr1; // Depoziteaza rezultatul blocat (eroare*mu) pe 'my0' - el se aplica la toti coeficientii
mr=mx0*my0(rnd), ay0=pm(i6,m5); // mr1 = x(n-k) * [eroare*mu] | Aduce primul h_vechi(k)
                                         // in ay0 pt viitor

do adaptive1 until ce; // se repeta blocul de instructiuni de aici pana la eticheta adaptive1
```

```

ar=mr1+ay0, mx0=dm(i3,m1), ay0=pm(i6,m4); // ar = Corectia + h_vechi(k). Paralel aduce
urmatorul element din vectorul date x si vectorul coeff
mf=mx1*my1(rnd), sr1=pm(i6,m7); // mf = lambda(mx1) * mu(my1). Aduce iarasi din PM in
shifter 'sr1' vechiul coeff
mr=sr1*mf(rnd); // mr1 = h(k) * [mu*lambda]. Acesta este Leakage-ul
ay1 = mr1; // Salvam Leakage-ul in ay1 pentru operatii ALU
ar=ar-ay1; // ar = (h(k) + corectia LMS_standard) - termen Leakage -> Obtinem Noul
Coeficient

adaptive1:
pm(i6,m6)=ar, mr=mx0*my0(rnd); // Suprascrim vechiul coeficient pm cu 'ar' (h_nou). Paralel
calculam noua corectie x(n-k)*[eroare*mu] in avans pt iteratia j+1
modify(i3,m3); // Dupa bucla se realigneaza/recupereaza pointerii circulari
modify(i6,m4);
rts; // Inapoi la fluxul ALE.

```

ALGORITMUL FILTRU MEDIAN

```

run_mf:
ar=dm(mf_flag_init); // Citire flag initializare
ar=ar-1;
if eq call mf_init; // Apel routine curățare buffer daca a fost proaspăt rulat (flag=1)
ar=dm(temp_in); // Aduce x(n) in 'ar'
dm(i3,m3)=ar; // Bagă x(n) in bufferul circular delay_mf, se incrementează i3
Căci algoritmul de sortare va distruge ordinea elementelor din timp, trebuie creata o
copie liniara temporara
i0=delay_sorted; // i0 va prelua scrierea pentru bufferul temporar ordonat
cntr=W_MF; // Setam contor = W (Lățimea ferestrei = 3)
do copy until ce; // Copiază fereastra (din delay circular -> liniar)
si=dm(i3,m3); // Citește vechi din ring buffer, se incrementează i3
copy:
dm(i0,m3)=si; // Baga noul in zona array temp. i0 face avans normal +1
// bufferul este circular, eşantioanele sunt stocate cronologic
i2=delay_sorted; // Acum zona e plina. i2 va fi folosit ca Base pointer in routinele de sortare
call sort_sel; // Apel subrutina Selection Sort pe arrayul temp

```

Extragerea valorii Mediane

```

i0=delay_sorted; // Cursorul pleaca din baza zonei temp ordonate
m0=K_MF; // Valoarea centrala a ferestrei este K=1 (K_MF). M_0 = pas shift offset.
modify(i0,m0); // Avansează i0 fix cu K poziții! i0 ajunge la elementul exact median.
ar=dm(i0,m3); // Extrage valoarea intr-un registru ALU
call write_out; // O trimite pe fir ca esantion procesat final!

rti; // Termina întreruperea

```

SORTARE PRIN SELECTIE (Rutina standard $O(n^2)$ aplicata ferestrei Median)

```
sort_sel:
```

Setare mediu DAG. Vectorii din sortare trebuie sa fie **OBLIGATORIU** parcursi liniar.

```
m3=1; // pas +1
```



```

l0=0; l1=0;          // Șterge flag-ul Buffer Circular de pe pointerii 0 si 1
cntr=W_MF;          // Încarcă in bucla mare 3 iterații

```

Baza buclei externe LOOP_I

```

do loop_i until ce;
ax0=W_MF; ay0=cntr; ar=ax0-ay0; // Trick de contor ascendent in DSP: i = 3 - counter_descrescator
m0=ar;                      // Variabila logica "min" se stocheaza in pointerul m0. Aici: m0 = min = i
m1=ar;                      // Variabila "i" (current index) se stocheaza pe m1. m1 = i
ar=ar+1;
m2=ar;                      // Setare "j start". m2 = i + 1. De unde pornește bucla interna!
ay0=m1;                      // preia i
ar=ax0-ay0;                  // nr_elemente = W - i
ar=ar-1;                      // nr_iteatii ramase = (W - i) - 1. Aflam de cate ori se învâрте Loop_J
if eq jump loop_i; // Daca s-a ajuns la ultimul i, bucla j nu are sens sa se miște (are 0 iterații), sari!
cntr=ar;                      // Încarcă counter pt Loop_J cu restul
i0=i2;                      // Întoarce i0 la Base Array delay_sorted
modify(i0,m2) // Si pune-l direct pe adresa lui A[i+1]

```

Baza buclei interne LOOP_J

```

do loop_j until ce;          // FOR J
ax0=dm(i0,m3);              // ax0 citește înainte "A[j]"
i1=i2;                      // pregătim citirea minimului relativ la Base Array
modify(i1,m0);              // adăugam la Base Array decalajul "min_idx". i1 se duce la A[min]
ay0=dm(i1,m3);              // ay0 ia valoarea găsită in memorie pt "A[min]"
ar=ax0-ay0;                  // Execuția logica hardware: ( A[j] - A[min] )
if gt jump loop_j; // Daca (A[j] - A[min]) e pozitiv (Greater Than) -> A[j] nu e cel mai mic, sari la nxt J.

// Daca codul a ajuns aici, am gasit o valoare mai mica! A[j] < A[min]
ax0=W_MF; ay0=cntr; ar=ax0-ay0; // Recuperam J index din counter
m0=ar;                      // Salvam j pe postul pointerului MINIM: min = j !!
loop_j: nop;                // Sfarsit Loop J (buclo interna)

```

Schimbarea valorilor SWAP in memorie:

```

i1=i2; modify(i1,m1);
ax1=dm(i1,m3);              // extragem ax1 (tmp) de la adresa Base + "i"
i1=i2; modify(i1,m0);
ay1=dm(i1,m3);              // extragem ay1 de la adresa Base + "min"
i1=i2; modify(i1,m1);
dm(i1,m3)=ay1;              // Suprascriem Base + "i" cu valoarea minima adusa in ay1
i1=i2; modify(i1,m0);
dm(i1,m3)=ax1;              // Suprascriem Base + "min" cu valoarea tmp reținuta in ax1
loop_i: nop;                // Sfârșit Loop I (buclo externa)
rts;                        // Intrare din sortare înapoi in MF

```

SCRIERE IESIRE

```

write_out:
    dm(temp_out) = ar;      // Registrul de prelucrare principal 'ar' se descarca in variabila temporara
                             temp_out

```

```

    ax0 = dm(canal);          // Verificam care e Canalul activ comandat de ARM (Stanga sau Dreapta)
    ay0 = 0; ar = ax0 - ay0; // Aflam starea
    if eq jump write_left; // Daca rezultatul e 0 (adica ax0=0), sare la comanda de scriere ptr canal
                           Stâng

write_right:
    ar = dm(temp_out);        // Preluare eşantion procesat
    dm(tx_buf + 2) = ar;      // Offset 2 reprezintă Cuvântul 3 din buffer-ul circular de transmisie
                           Canal Dreapta)
    rts;                      // Final rutine IO

write_left:
    ar = dm(temp_out);
    dm(tx_buf + 1) = ar;      // Offset 1 reprezintă Cuvântul 2 (Canal Stânga)
    rts;

```

RUTINE AUXILIARE

Selecție Citire Canal Input

```

read_buff:
    ax0 = dm(canal); ay0 = 0; ar = ax0 - ay0; // Verificam canal activ (0 sau 1)
    if eq jump c0;                          // Daca canal e 0, sare la stânga
    ay0 = 1; ar = ax0 - ay0;
    if eq jump c1;                          // Daca canal e 1, sare la dreapta
c0:
    ar = dm(rx_buf + 1); rts;                // Trage datele nealterate din rx_buf slot stânga
c1:
    ar = dm(rx_buf + 2); rts;                // Trage din rx_buf slot Dreapta

```

Rutina Inițiere Codec

Se executa imediat după boot in interrupt Tx pt a configura AD1847 secvențial

```

next_cmd:          //trimiterea efectivă a celor 13 comenzi de configurare și oprirea fazei de
                   inițializare
    ena sec_reg; ax0 = dm(i3, m1); dm(tx_buf) = ax0; // Trage comanda hardware spre buffer_tx
                   folosind pt paralel
    ax0 = i3; ay0 = init_cmds; ar = ax0 - ay0; if gt rti; // Compara adresa curenta i3 cu adresa de baza
                   inițiala
    ax0 = 0xaf00; dm(tx_buf) = ax0; ax0 = 0; dm(stat_flag) = ax0; rti; // Închide setul după executarea
                   celor 13 cadre hardware

```

Inițializare AGC

```

agc_init:
    si=0;
    dm(S_agc)=si;          // Resetează Media
    dm(g_fr_agc)=si;        // Resetează fracționar part g(n-1)
    dm(g_int_agc)=si;       // Resetează integer part g(n-1)
    si=0.5r;                // DSP știe reprezentarea float fix '0.5r' convertindu-l automat
    dm(ref_agc)=si;         // Stabilește Referința default = 0.5
    si=0.05r;
    dm(mu_agc)=si;          // Pasul de învățare mu=0.05

```

```

se=-K_AGC; // Pregătește Shift divizor
l3=M_AGC; i3=delay_agc; m3=1; // Resetează Pointerul Istoricului pt a evita erori de citire
si=0;
dm(agc_flag_init)=si; // Flag init dezactivat
rts;

```

Inițializare MF

```

mf_init:
l3=W_MF; i3=delay_mf; m3=1; l0=0; // Setup pt median, șterge buffer offset
si=0; dm(mf_flag_init)=si; // Dezactivează request-ul de clear
rts;

```

Inițializare ALE

Inițializări constante

```

ale_init:
si=0.01r;
dm(mu_ale) = si; // Viteza algoritmu
dm(lambda_ale) = si; // Viteza leakage
m1=1; m5=1;
si=0;

```

Re-aloca DAG pointers pentru lungimile bufferelor circulare si adrese baza

```

i2=input_ale; l2=N_ALE;
i4=input_ale+D_ALE; l4=N_ALE;
i3=fir_d; l3=N_ALE;
i6=hh; l6=N_ALE;

```

Scoate eșantioanele anterioare atât din bufferul de intrare cat si din linia de intarziere filtrului

```

cntr=N_ALE;
do init_buf until ce; // Bucla ce impinge 256 de zerouri succesiv
dm(i2,m1)=si; // in input memory
dm(i3,m1)=si; // in filtrul mem

```

Bucla este înlănțuita cu o instrucțiune de memorie program (curata si vechii coeficienți hh)

```

init_buf:
pm(i6,m5)=si; // in program memory buffer
si=0;
dm(ale_flag_init)=si; // Clear the Flag
rts;

```

IV. Setarea parametrilor funcției dsp conform stării switch-urilor de pe portul de intrare

Declararea adresei portului de intrare și valorilor maxime ale parametrilor

```
#define PORT_IN 0x1FF
#define M_AGC_MAX 128
#define N_ALE_MAX 256
#define W_MF_MAX 17
```

M_AGC_MAX, N_ALE_MAX și W_MF_MAX reprezintă valorile maxime regăsite în vectorii din care își setează variabilele M_AGC, N_ALE, W_MF valoarea. Definim această valoare maximă pentru alocare fixă de memorie în cazul vectorilor din program a căror dimensiune depinde de acești parametri.

```
/* in .SECTION/DM      data1;*/
```

```
.var/circ delay_agc[M_AGC_MAX];
```

```
.var/circ delay_mf[W_MF_MAX];
```

```
.var delay_sorted[W_MF_MAX];
```

```
.var/circ fir_d[N_ALE_MAX];
```

```
.var/circ input_ale[N_ALE_MAX];
```

```
/* in .SECTION/PM      pm_da;*/
```

```
.var/circ hh[N_ALE_MAX];
```

Inlocuirea parametrilor declarati #define cu variabile .var

Am declarat variabile de tip vector care să stocheze valorile posibile pentru fiecare parametru variabil al fiecărei funcții. Valoarea regăsită pe SWITCH-URILE aferente unui parametru va reprezenta indexul din vector al valorii pe care o va lua parametrul.

Valorile default ale parametrilor programului sunt cele de pe poziția 0 în fiecare vector de valori.

În cazul funcției **AGC**, am considerat că cele 8 SWITCH-URI ne vor da informații despre cei 3 parametri M_AGC, mu_agc și ref_agc (pentru că K_AGC se determină din valoarea lui M_AGC) astfel:

SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
ref_agc	ref_agc	mu_agc	mu_agc	mu_agc	M_AGC	M_AGC	M_AGC

```
.SECTION/DM      data1;
```

```
/* in Variabile AGC*/
```

```
.var M_AGC;
```

```
.var K_AGC;
```

```
.var ref_agc;
```

```
/* Nivelul de Referinta (R) */
```

```
.var mu_agc;
```

```
/* Pasul de adaptare (m) */
```

```
.var val_M_AGC[8] = 4, 1, 2, 8, 16, 32, 64, 128; // K va fi  $\log_2 M$ 
```

```
.var val_K_AGC[8] = 2, 0, 1, 3, 4, 5, 6, 7; // mai ușor de parcurs vectorul decât de impl  $\log_2$ 
```

```
.var val_mu_agc[8] = 0.05r, 0.0r, 0.001r, 0.015r, 0.01r, 0.1r, 0.5r, 0.9r;
```

```
.var val_ref_agc[4] = 0.5r, 0.2r, 0.7r, 0.95r;
```

În cazul funcției **MF**, am considerat că cele 8 SWITCH-URI ne vor da informații despre fereastra W_MF (pentru că K_MF se determină din valoarea lui W_MF, prin shiftare) astfel:

SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
					W_MF	W_MF	W_MF

/ in Variabile MF*/*

```
.var W_MF;
.var K_MF;
.var val_W_MF[8] = 3, 5, 7, 9, 11, 13, 15, 17;      // K va fi W>>1, nu este nevoie de vector suplim
```

În cazul funcției **ALE**, am considerat că cele 8 SWITCH-URI ne vor da informații despre cei 4 parametri N_ALE, D_ALE, mu_ale și lambda_ale astfel:

SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
N_ALE	N_ALE	D_ALE	D_ALE	mu_ale	mu_ale	lambda_ale	lambda_ale

/ in Variabile ALE*/*

```
.var N_ALE;
.var D_ALE;
.var mu_ale;
.var lambda_ale;
.var val_N_ALE[4] = 256, 20, 30, 128;
.var val_D_ALE[4] = 128, 5, 50, 256;
.var val_mu_ale[4] = 0.01r, 0.001r, 0.05r, 0.1r;
.var val_lambda_ale[4] = 0.01r, 0.001r, 0.1r, 0.5r;
```

Declararea unei variabile care stocheaza valoarea de la adresa portului de intrare

/ in Variabile control*/*

```
.var port_intrare;
/* in ISR input_samples (se citește la întrerupere asincronă)*/
ax0=IO(PORT_IN);
ay0=0x00FF;      //păstrăm ultimii 8 biți
ar=ax0 and ay0;
dm(port_intrare)=ar;
```

Prelucrarea valorii de pe portul de intrare pentru obținerea parametrilor

Pentru unii parametri (mu_agc, ref_agc, mu_ale, lambda_ale) am considerat că ar fi interesantă observarea actualizării valorii lor din mers și modificărilor aduse de această tranziție asupra semnalului de intrare. Astfel, citirea biților corespondenți lor și actualizarea lor se face la fiecare eșantion prelucrat, și sunt incluse în subrutinele de apelare a funcțiilor, run_agc și run_ale. În cazul celorlalți parametri, M_AGC, K_AGC, N_ALE, D_ALE și W_MF, am considerat suficientă citirea lor la întrerupere asincronă, în consecință citirea biților corepondenți lor a fost integrată în inițializările funcțiilor, care sunt de asemenea declanșate la întrerupere asincronă (și dacă filtrul a fost schimbat).

/ la începutul run_agc, după (re)setarea agc_init */*

```
ax0=dm(port_intrare);      //valoarea citită de pe SW7-0
//mu_agc
ay0=0x0038;                // izolăm biții care dau valoarea pentru mu_agc
ar=ax0 and ay0;            // mască cu 0011 1000 => păstrez b5,b4,b3
se = -3;                   // shift dreapta cu o val care să îmi aducă biții de interes pe poziții nesemnif
sr = lshift ar (lo);        // din 00b5b4 b3000 obțin 0000 0b5b4b3

i0 = val_mu_agc;           // pointerul i0 indică prima adresă din vectorul de valori val_mu_agc
m0 = sr0;                  // pasul va fi valoarea biților b5b4b3 aduși pe pozițiile cele mai nesemnif
modify(i0, m0);            // pointerul se deplasează pe poziția corespondentă valorii biților b5b4b3

ax1 = dm(i0, m0);          // aducem în ALU valoarea din memorie indicată de pointerul i0
                             // actualizat cu pasul m0
dm(mu_agc) = ax1;          //încărcăm în variabila din DM mu_agc valoarea din registrul ax1 din ALU

//ref_agc
ay0=0x00C0;                // izolăm biții care dau valoarea pentru ref_agc
ar=ax0 and ay0;            // mască cu 1100 0000 => păstrez b7 și b6
se = -6;                   // shift dreapta cu o val care să îmi aducă biții de interes pe poziții nesemnif
sr = lshift ar (lo);        // din b7b600 0000 obțin 0000 00b7b6

i0=val_ref_agc;            // pointerul i0 indică prima adresă din vectorul de valori val_ref_agc
m0=sr0;                    // pasul va fi valoarea biților b7b6 aduși pe pozițiile cele mai nesemnif
modify(i0, m0);            // pointerul se deplasează pe poziția corespondentă valorii biților b7b6

ax1=dm(i0, m0);            // aducem în ALU valoarea din memorie indicată de pointerul i0
                             // actualizat cu pasul m0
dm(ref_agc) = ax1;         //încărcăm în variabila din DM ref_agc valoarea din registrul ax1 din ALU
```

/ la începutul agc_init */*

```
//M_AGC și K_AGC
ax0=dm(port_intrare);      //valoarea citită de pe SW7-0
ay0=0x0007;                // izolăm biții care dau valoarea pentru M_AGC
ar=ax0 and ay0;            // mască cu 0000 0111 => păstrez b2,b1,b0
                             //nu mai e nevoie de shiftare la dreapta, biții de interes sunt deja pe poz nesemnif
i0=val_M_AGC;              // pointerul i0 indică prima adresă din vectorul de valori val_M_AGC
m0=ar;                     // pasul va fi valoarea biților b2b1b0
modify(i0, m0);            // pointerul se deplasează pe poziția corespondentă valorii biților b2b1b0
ax1=dm(i0, m0);            // aducem în ALU valoarea din memorie indicată de pointerul i0
                             // actualizat cu pasul m0
dm(M_AGC)=ax1;             //stocăm valoarea aflată din vectorul de valori în variabile

i0=val_K_AGC;
modify(i0, m0);            //pentru K_AGC parcurgem același pas spre valoarea din vectorul de valori
ax1=dm(i0, m0);            //pentru că valorile sale și ale M_AGC de pe aceeași poziție corespund
dm(K_AGC)=ax1;
```

/ la începutul run_ale */*

```
ax0=dm(port_intrare);      //valoarea citită de pe SW7-0
//lambda_ale
ay0=0x0003;                // izolăm biții care dau valoarea pentru lambda_ale
ar=ax0 and ay0;            // mască cu 0000 0011 => păstrez b1,b0, nu e nevoie de shiftare dreapta
i0=val_lambda_ale;        //pointerul i0 indică prima adresă din vectorul de valori val_lambda_ale
m0=ar;                     // pasul va fi valoarea biților b1b0
modify(i0, m0);           // pointerul se deplasează pe poziția corespondentă valorii biților b1b0
ax1=dm(i0, m0);           //aducem în ALU valoarea din memorie indicată de pointerul i0 actualizat cu m0
dm(lambda_ale)=ax1;       //încărcăm în variabila din DM lambda_ale valoarea din registrul ax1 din ALU
//mu_ale
ay0=0x000C;                // izolăm biții care dau valoarea pentru mu_ale
ar=ax0 and ay0;            // mască cu 0000 1100 => păstrez b3,b2
se=-2;                     //shiftez dreapta cu o val care să îmi aducă biții de interes pe poziții nesemnif
sr = lshift ar (lo);       // din 0000 b3b200 obțin 0000 00b3b2
i0=val_mu_ale;             // pointerul i0 indică prima adresă din vectorul de valori val_mu_ale
m0=sr0;                    // pasul va fi valoarea biților b3b2 aduși pe pozițiile cele mai nesemnif
modify(i0, m0);           // pointerul se deplasează pe poziția corespondentă valorii biților b3b2

ax1=dm(i0, m0);           //aducem în ALU valoarea din memorie indicată de pointerul i0 actualizat cu m0
dm(mu_ale)=ax1;           //încărcăm în variabila din DM mu_ale valoarea din registrul ax1 din ALU
```

/ la începutul ale_init */*

```
ax0=dm(port_intrare);      //valoarea citită de pe SW7-0
//D_ALE
ay0=0x0030;                // izolăm biții care dau valoarea pentru D_ALE
ar=ax0 and ay0;            // mască cu 0011 0000 => păstrez b5,b4
se=-4;                     //shiftez dreapta cu o val care să îmi aducă biții de interes pe poziții nesemnif
sr = lshift ar (lo);       // din 00b5b40000 obțin 0000 00b5b4

i0=val_D_ALE;              // pointerul i0 indică prima adresă din vectorul de valori val_D_ALE
m0=sr0;                    // pasul va fi valoarea biților b5b4 aduși pe pozițiile cele mai nesemnif
modify(i0, m0);           // pointerul se deplasează pe poziția corespondentă valorii biților b5b4

ax1=dm(i0, m0);           //aducem în ALU valoarea din memorie indicată de pointerul i0 actualizat cu m0
dm(D_ALE)=ax1;            //încărcăm în variabila din DM D_ALE valoarea din registrul ax1 din ALU
//N_ALE
ay0=0x00C0;                // izolăm biții care dau valoarea pentru N_ALE
ar=ax0 and ay0;            // mască cu 1100 0000 => păstrez b7,b6
se=-6;                     //shiftez dreapta cu o val care să îmi aducă biții de interes pe poziții nesemnif
sr = lshift ar (lo);       // din b7b600 0000 obțin 0000 00b7b6

i0=val_N_ALE;              // pointerul i0 indică prima adresă din vectorul de valori val_N_ALE
m0=sr0;                    // pasul va fi valoarea biților b7b6 aduși pe pozițiile cele mai nesemnif
modify(i0, m0);           // pointerul se deplasează pe poziția corespondentă valorii biților b7b6

ax1=dm(i0, m0);           //aducem în ALU valoarea din memorie indicată de pointerul i0 actualizat cu m0
dm(N_ALE)=ax1;            //încărcăm în variabila din DM N_ALE valoarea din registrul ax1 din ALU
```


/ la începutul mf_init */*

```
ax0=dm(port_intrare);      //valoarea citită de pe SW7-0
//W_MF
ay0=0x0007;                // izolăm biții care dau valoarea pentru W_MF
ar=ax0 and ay0;            // mască cu 0000 0111 => păstrez b2,b1,b0 ,nu e nevoie de shiftare dreapta

i4=val_W_MF;               // pointerul i4 indică prima adresă din vectorul de valori val_W_MF
m4=ar;                     // pasul va fi valoarea biților b2b1b0
modify(i4, m4);            // pointerul se deplasează pe poziția corespundentă valorii biților b2b1b0

ax1=dm(i4, m4);            // aducem în ALU valoarea din memorie indicată de pointerul i4
dm(W_MF)=ax1;              //stocăm valoarea extrasă din vector în variabila DM W_MF

//K_MF
se=-1;                     //K_MF obț prin shiftarea la dreapta cu 1 a valorii stocate în variabila W_MF
ar=ax1;                    // aducem în ALU valoarea lui W_MF
sr=lshift ar (lo);
dm(K_MF)=sr0;              //încărcăm în variabila din DM K_MF valoarea obținută
```

În subrutinele run_agc, run_ale am folosit pentru actualizarea parametrilor registre DAG care să nu suprascrie la fiecare eșantion registrele DAG folosite de algoritm.

Modificări survenite în cod

În codul fiecărei funcții, acolo unde sintaxa pentru parametrii declarați până acum #define a fost, de exemplu,

```
ax0=W_MF;    va deveni    ax0=dm(W_MF);

cntr=N_ALE           cntr=dm(N_ALE); //încărcarea în registrul de control cntr se
                        // poate face din DM (adresare directă)

cntr=M_AGC-1;        ar=dm(M_AGC);
                        ar=ar-1; //operanzii din DM sunt încărcăți întâi în ALU, apoi prelucrați
                        cntr=ar; //încărcarea rezultatului în contor

m0=K_MF;             m0=dm(K_MF); //încărcarea în registrele DAG se face direct din DM

i4=input_ale+D_ALE    m4= dm(D_ALE);
                        i4=input_ale;
                        modify(i4, m4); //actualizarea pointerului cu un număr de pași dat de
                        // valoarea stocată în variabila D_ALE din memoria de date
```

K_AGC va fi încărcat în se în run_agc și nu în agc_init pentru că nu vrem să-l pierdem prin shiftările de prelucrare a valorii de pe portul de intrare.

```
ax0=dm(K_AGC);        //aduc valoarea din memorie într-un registru ALU
ar=-ax0;               //facem negarea in ALU
se=ar;                 //salvez rezultatul in shift exponent
Cod echivalent cu linia se=-K_AGC; a codului anterior.
```

V. Scriere la adresa portului de ieșire

Disponem de un afișor cu 7 segmente care trebuie să afișeze starea internă a programului. În consecință, afișorul va afișa cifra asociată algoritmului filtrului care rulează în prezent.

Afișor cu 7 segmente

Acest dispozitiv de afișare este format, așa cum sugerează și numele, din 7 segmente ce pot fi controlate separat. Putem aprinde sau stinge fiecare segment controlând tensiunea aplicată pe acesta. Pentru acest afișaj cu anod comun, comanda este în logică negativă: aplicând 0 logic pe un segment acesta se va aprinde, iar aplicând 1 logic, acesta se va stinge.

AGC – 00 (0)

Pentru a sugera că funcția AGC este executată în prezent, pe afișor vom vrea să aprindem segmentele 0, 1, 2, 3, 4 și 5 => 100 0000

=> 0x40 pentru AGC

ALE – 01 (1)

Pentru a sugera că funcția ALE este executată în prezent, pe afișor vom vrea să aprindem segmentele 1 și 2 => 111 1001

=> 0x79 pentru ALE

MF – 10 (2)

Pentru a sugera că funcția MF este executată în prezent, pe afișor vom vrea să aprindem segmentele 0, 1, 3, 4 și 6 => 010 0100

=> 0x24 pentru MF

Comandă invalidă – 11 (3)

Pentru a sugera că în prezent se execută out=in, pe afișor vom vrea să aprindem segmentele 0, 1, 2, 3 și 6 => 011 0000

=> 0x30 pentru comandă invalid



Declararea adresei portului de ieșire și vectorului de valori pentru segmentele afișorului

```
#define PORT_OUT 0xFF
```

```
.SECTION/DM          data1;
```

```
/* în --- Variabile de Control --- */
```

```
.var TAB_AFIS[4] = 0x40, 0x79, 0x24, 0x30;           // afișez 0, 1, 2 sau 3 în funcție de filtru
```

Vectorul de valori prin care se aprind/sting segmentele afișorului conține cele 4 valori aflate anterior. Indexul fiecărei valori din vector este identic cu valoarea cifrei care se va afișa, și totodată cu valoarea zecimală a codului funcției.

Implementarea și integrarea în cod a subrutinei de afișare

Rutina de afișare se va apela la fiecare întrerupere asincronă și dacă `cda != cda_prev`, și în interiorul subrutinei de comandă invalidă.

```
// în input_samples, înainte de cont: //
```

```
// În si este salvată valoarea dm(cda).
```

```
call afisare;
```

```
invalid_cmd:
```

```
    call read_buff;
```

```
    si=3;                //ne asigurăm că în si se află valoarea pe care vrem să o regăsim pe afișor
```

```
    call afisare;
```

```
    jump write_out;
```

```
/* în Rutine Auxiliare */
```

```
/* Afisor */
```

```
afisare:
```

```
    i0=TAB_AFIS;        //Pointerul arată către prima adresă din vectprul TAB_AFIS
```

```
    m0=si;
```

```
                        //Pointerul înaintează cu numărul de pași dat de valoarea zecimală a
```

```
    modify(i0, m0);     // codului funcției prin vectorul de valori
```

```
    si=dm(i0, m0);     //Încărcăm în si valoarea regăsită la adresa la care a ajuns pointerul
```

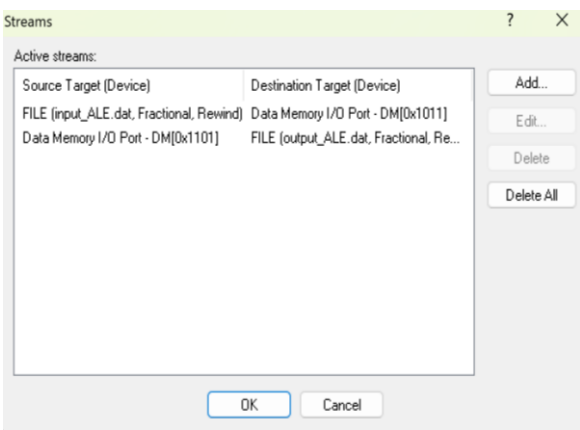
```
    IO(PORT_OUT)=si;   //Punem acea valoare la adresa portului de ieșire
```

```
    rts;
```

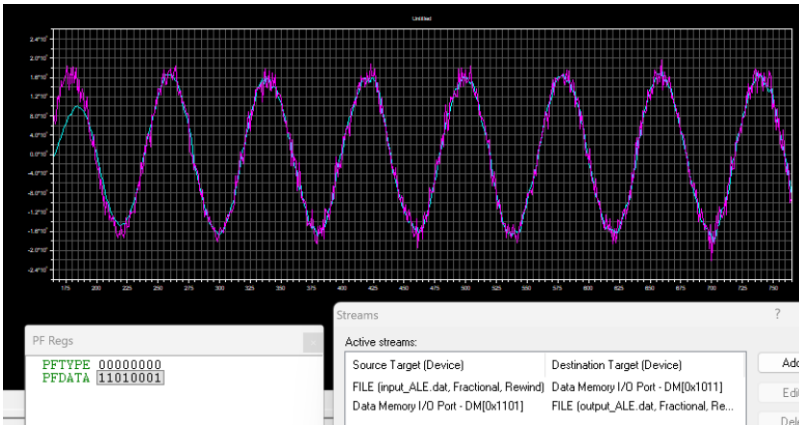
VI. Simulări

Algoritm	Canal	Hex	Binar	D7D6 D5D4 D3 D2D1D0
AGC	Stânga	0xC1	1100 0001	11 00 0 001
AGC	Dreapta	0xC9	1100 1001	11 00 1 001
ALE	Stânga	0xD1	1101 0001	11 01 0 001
ALE	Dreapta	0xD9	1101 1001	11 01 1 001
MF	Stânga	0xE1	1110 0001	11 10 0 001
MF	Dreapta	0xE9	1110 1001	11 10 1 001
BYPASS (invalid)	—	orice	D7 sau D6 = 0	—

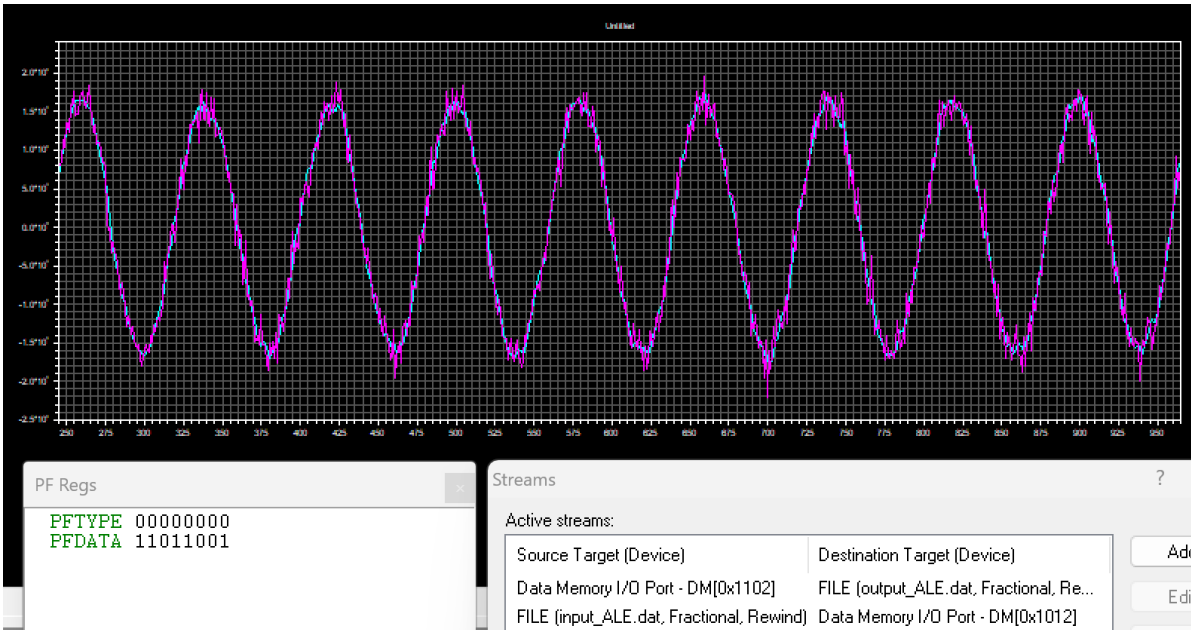
Adaptive Line Enhancer (ALE)



Streamuri canal stânga

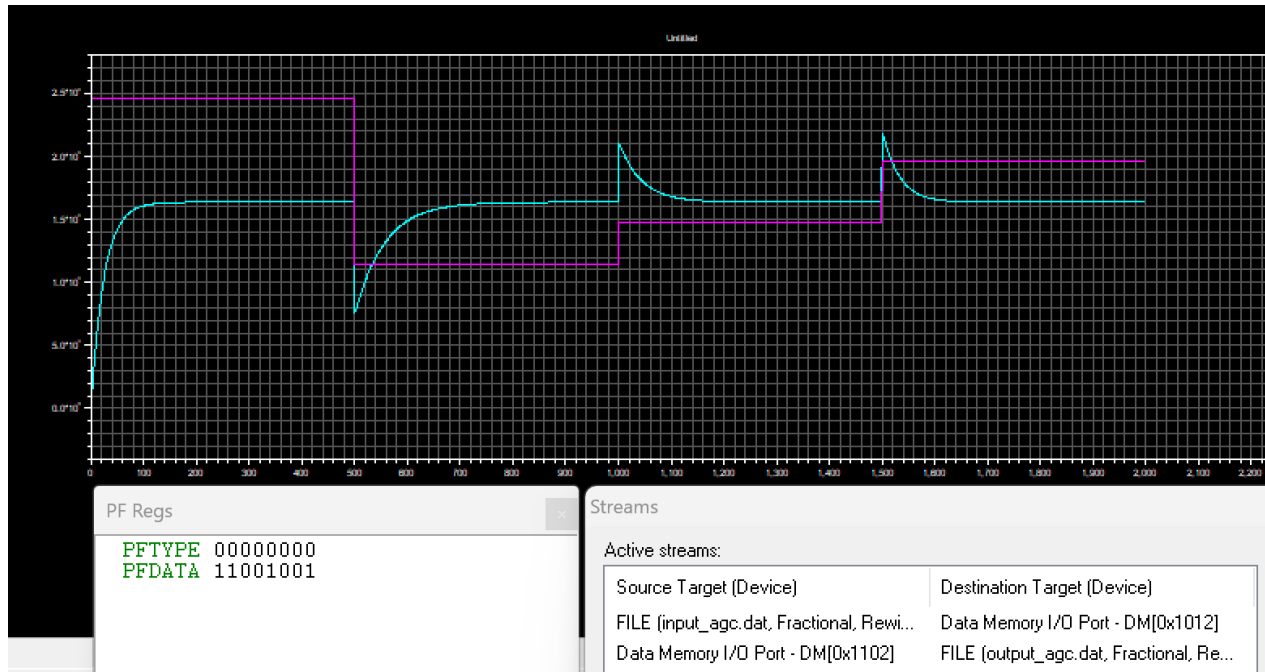


Semnal intrare&iesire canal stanga

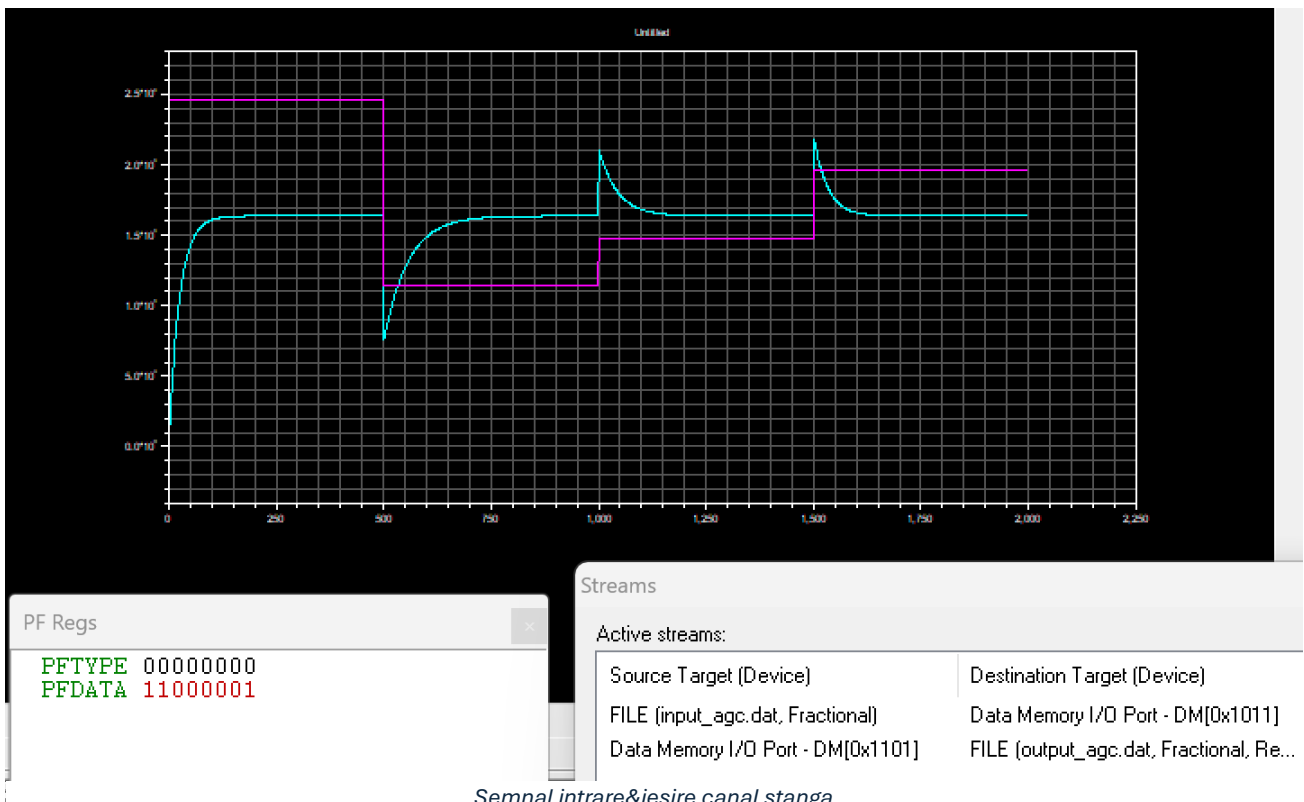


Semnal intrare&iesire canal dreapta

Automatic Gain Control (AGC)

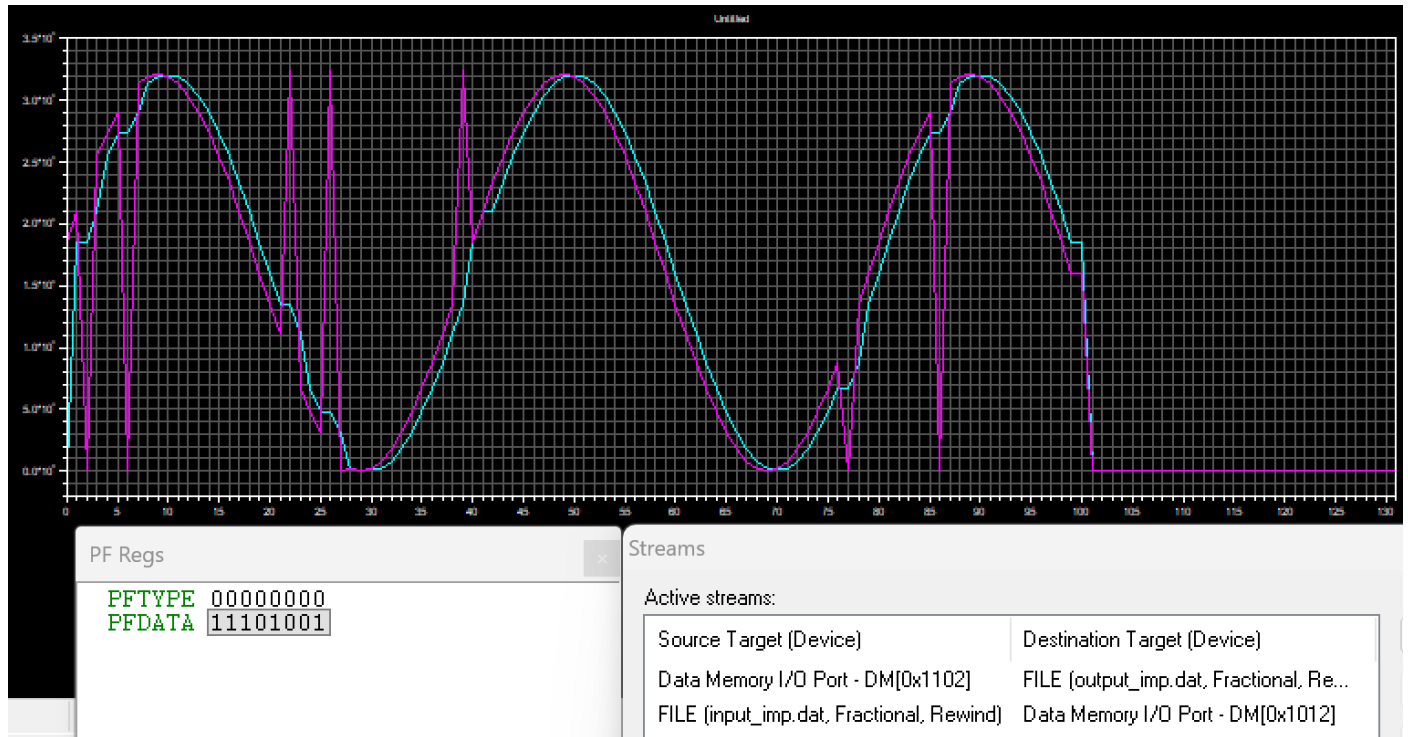


Semnal intrare&iesire canal dreapta

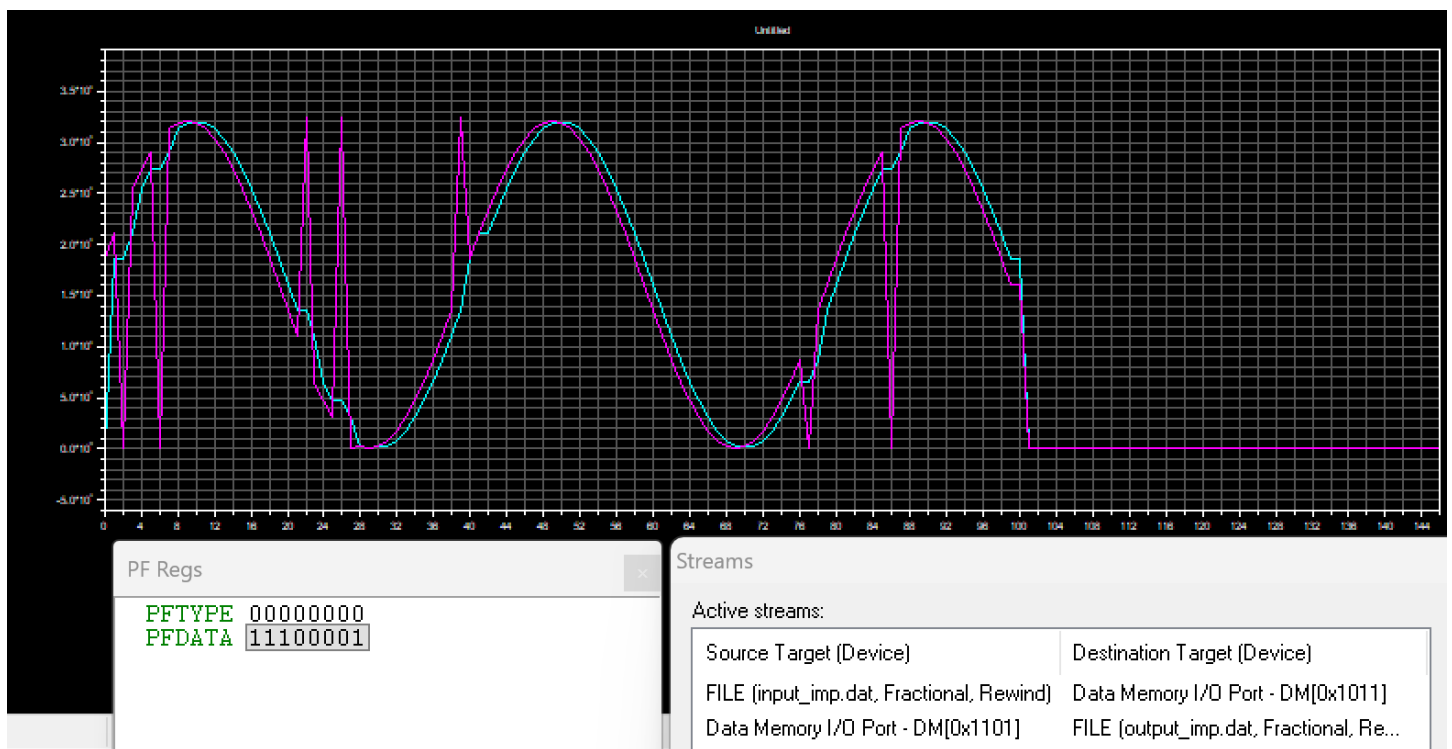


Semnal intrare&iesire canal stanga

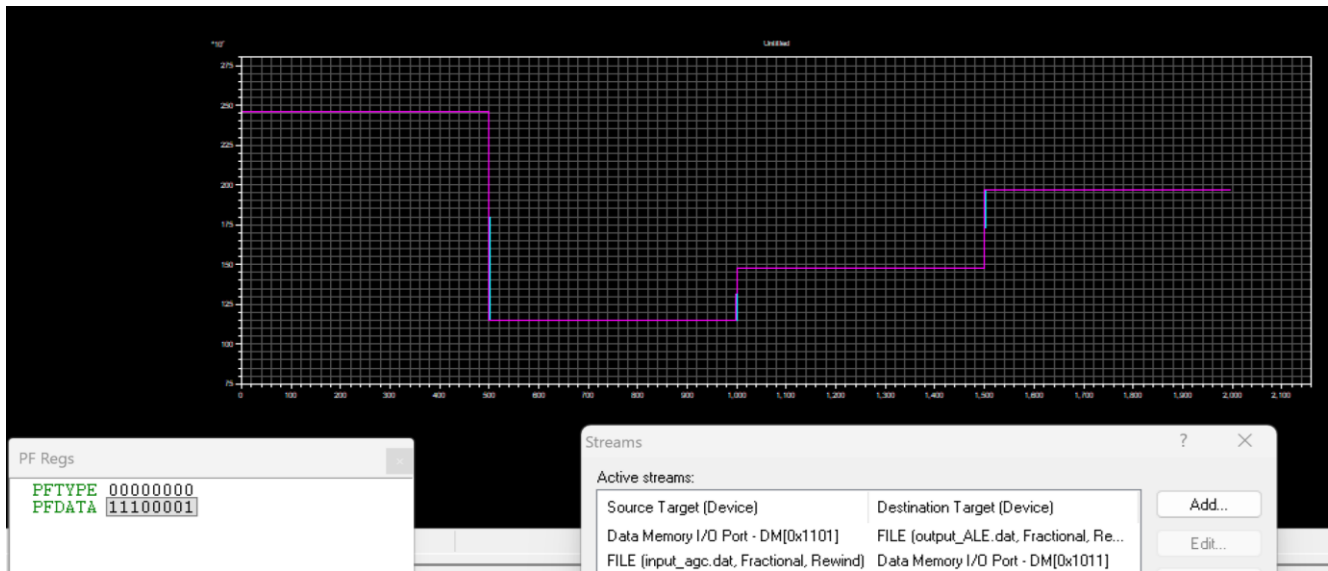
Median Filter (MF)



Semnal intrare&iesire canal dreapta



Semnal intrare&iesire canal stanga



Aplicare MF pe semnal AGC

VII. Realizarea fizică a sistemului STM-DSP

STM32

Conectarea butoanelor

Pentru butoanele B7-B0 (*PA8-PA1*), am activat rezistențe interne de pull-down pentru a stabili starea pinilor asociați la 0 logic atunci când butonul nu este apăsat.

Pentru PRG, am activat rezistență internă de pull-up pentru a marca apăsarea butonului prin trecerea în starea de 0 logic a pinului *PA9* (neapăsarea butonului echivalentă cu starea de 1 logic prin rezistență).

The image shows two screenshots from the STM32CubeMX software. The left screenshot displays the 'GPIO Mode and Configuration' window. It shows a table of pin configurations for PA1 through PA9. PA1-PA8 are configured as Input mode with Pull-down, and PA9 is configured as Input mode with Pull-up. The right screenshot shows the 'Pinout view' of the STM32F103CBT_x LQFP48 package. It highlights the pins PA1 through PA9 and their corresponding functions: PA1 (B0), PA2 (B1), PA3 (B2), PA4 (B3), PA5 (B4), PA6 (B5), PA7 (B6), PA8 (B7), and PA9 (PRG).

Pin Na...	Signal on	GPIO outp...	GPIO mode	GPIO Pull...	Maximum...	User Label	Modified
PA1	n/a	n/a	Input mode	Pull-down	n/a	B0	✓
PA2	n/a	n/a	Input mode	Pull-down	n/a	B1	✓
PA3	n/a	n/a	Input mode	Pull-down	n/a	B2	✓
PA4	n/a	n/a	Input mode	Pull-down	n/a	B3	✓
PA5	n/a	n/a	Input mode	Pull-down	n/a	B4	✓
PA6	n/a	n/a	Input mode	Pull-down	n/a	B5	✓
PA7	n/a	n/a	Input mode	Pull-down	n/a	B6	✓
PA8	n/a	n/a	Input mode	Pull-down	n/a	B7	✓
PA9	n/a	n/a	Input mode	Pull-up	n/a	PRG	✓

PA9 Configuration:

GPIO mode: Input mode

GPIO Pull-up/Pull-down: Pull-up

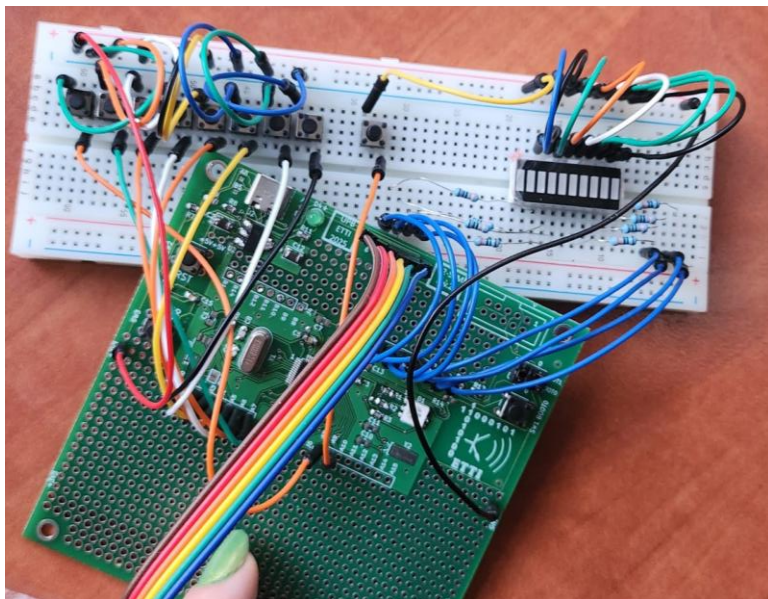
User Label: PRG

Un terminal al butoanelor va fi conectat la pinul de pe STM32 a cărui stare o determină. Celălalt terminal al butoanelor va fi conectat printr-un fir la 3.3V (*B7-B0*), respectiv la GND (*PRG*).

Conectarea LED-urilor

LED-urile au fost conectate cu catodul la GND, și anodul înseriat cu câte o rezistență de 330Ω (pentru limitarea curentului prin LED) spre pinul căruia îi corespund: *PB7-PB0*.

Am realizat legăturile pentru butoane și LEDuri încât să putem apăsa pe butoane și citi pe LEDuri formatul binar Little Endian, de la cel mai semnificativ bit (în stânga), până la cel mai nesemnificativ (în dreapta).

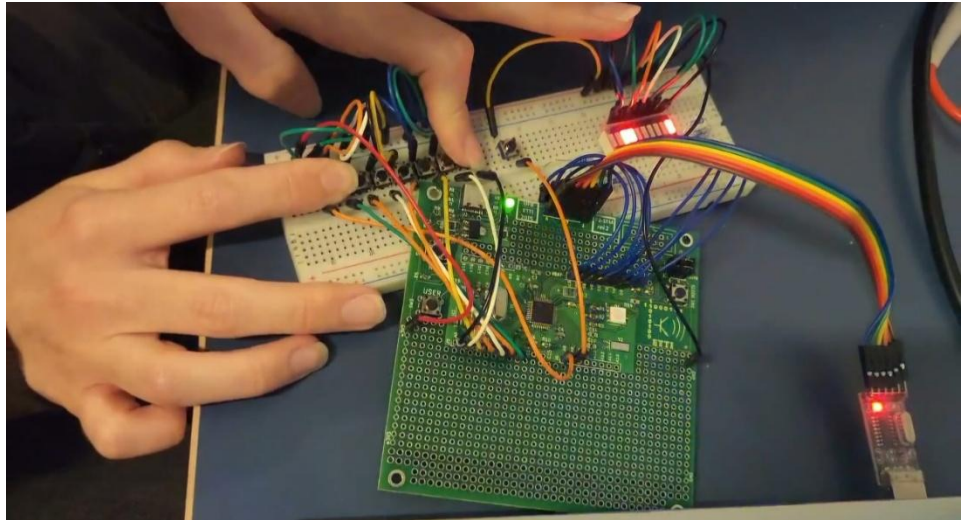


Funcționare

Generarea comenzii valide:

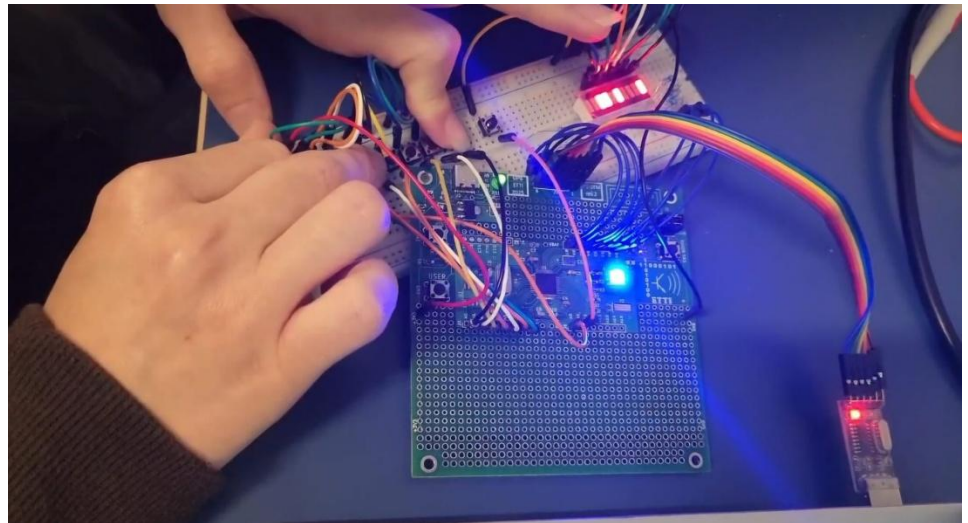
- pentru AGC canal stânga

11 00 0 001



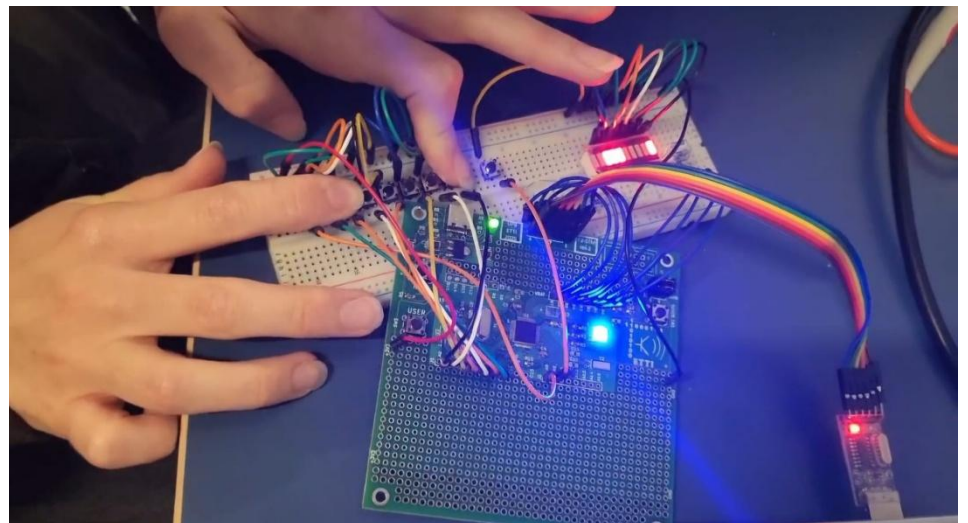
- pentru ALE canal stânga

11 01 0 001



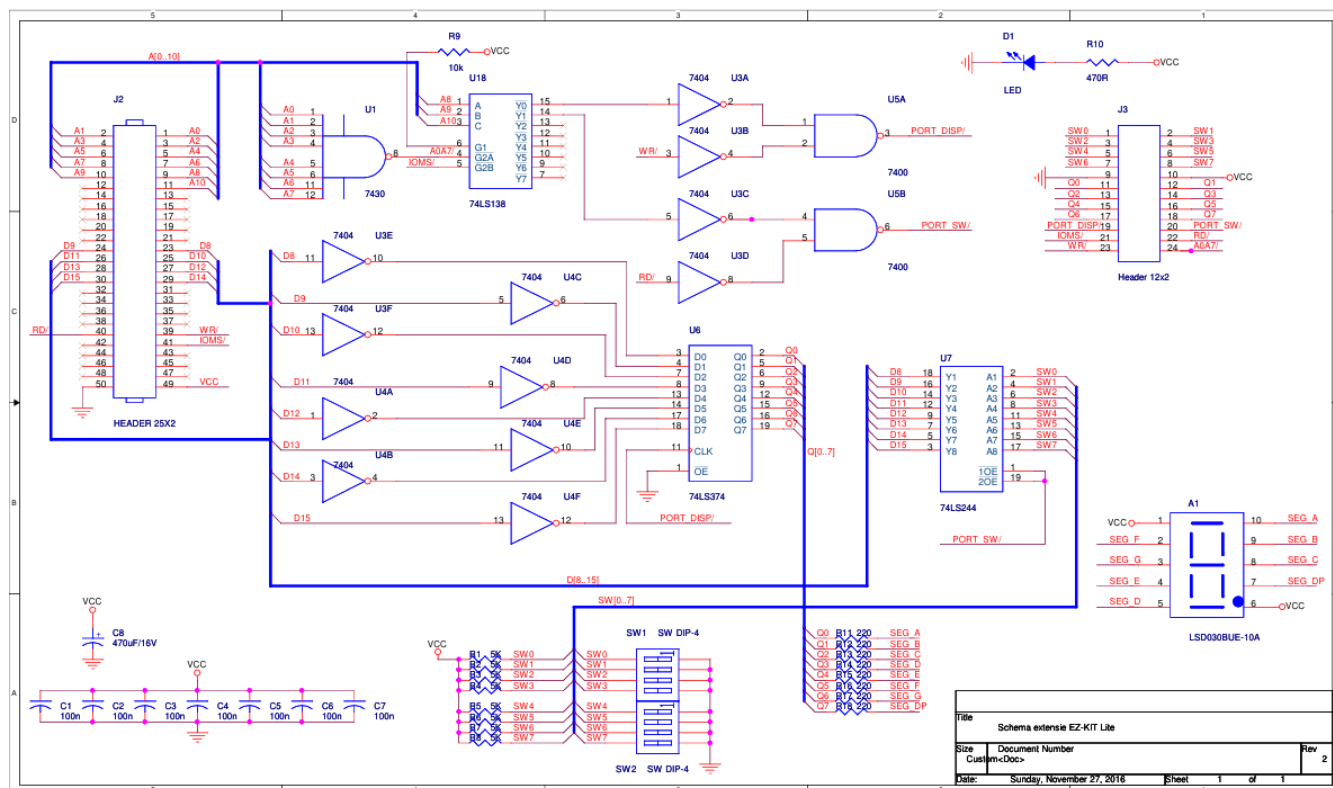
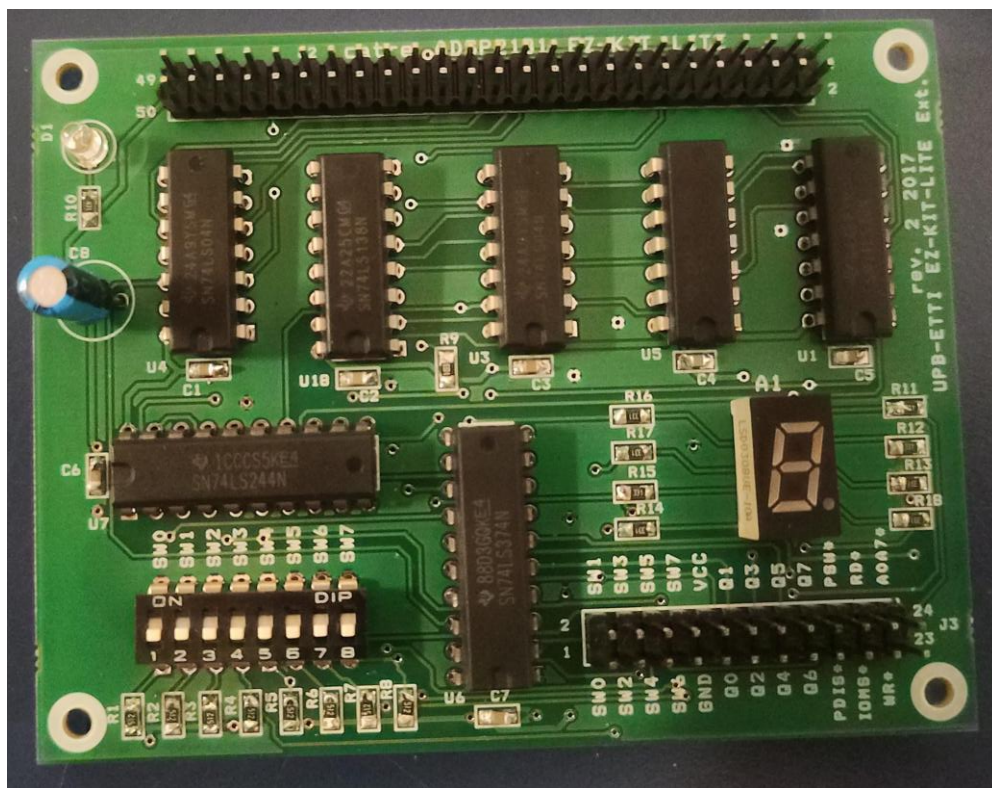
- pentru MF canal stânga

11 10 0 001



Placă de extensie I/O ADSP2181

Plasarea componentelor SMD și THT pe placa de extensie I/O (printre care switch-urile care determină parametrii funcțiilor și afișorul stării interne cu 7 segmente) conform schemei electrice



Schema electrică a plăcii de extensie I/O ADSP2181